What Checks the Proof?

A Textbook Introduction to Automated Mathematical Logic Deciders

BANK, FUTUREBANK, Imagination, Compass, Verification,
Controlled Creativity, and Formalized Self-Awareness

Textbook Draft v0.8

Brian Tenneson

August 21, 2026

Foreword

This book is about a particular question that appears simple until one tries to build a machine around it:

What checks the proof?

The question is not merely philosophical. It is architectural. A program may be brilliant at proposing lemmas, choosing search directions, imagining counterfactual proof routes, or even reasoning about its own strategy. None of those abilities, by themselves, tell us which mathematical claims are allowed to become established facts inside the machine.

The central object of the book is an *Automated Mathematical Logic Decider*, abbreviated AMLD. An AMLD is a certificate-producing architecture designed to settle a mathematical statement relative to a declared formal environment. Its four principal search roles are a prover P, a refuter R, a prover of independence I, and a prover of contradiction C. They do not merely race in isolation. They share a typed, provenance-preserving verified memory called the **BANK**. A separate **FUTUREBANK** represents possible proof futures that may be useful for planning before they are fully certified. A **COMPASS** ranks directions through that imagined future. A controller allocates attention. A verifier V is the fixed correctness boundary.

A distinction used throughout this edition is easy to state and important to preserve: *the role that discovers or submits a certificate is provenance; the semantic kind of the certificate says what it establishes.* A lemma discovered by R does not become an intrinsically "refuting" lemma, and a proof of c discovered while I is exploring a model-theoretic route is still a proof of c. Search-role tags record who found or submitted an object. Certificate semantics determine what the verifier may conclude from it.

The purpose of an AMLD is therefore broader than ordinary one-sided theorem proving. Given a target c relative to a background axiom or hypothesis set Γ, it may try to produce one of four kinds of positive evidence:

$$\Gamma \vdash c, \qquad \Gamma \vdash \neg c, \qquad \Gamma \nvdash c \text{ and } \Gamma \nvdash \neg c, \qquad \Gamma \vdash \bot.$$

The third outcome requires a genuine metamathematical certificate; failure to find proofs on the first two sides is not enough. The fourth outcome says that the admitted background has failed an integrity test. If no appropriate certificate is found within the available resources, the honest result is UNKNOWN.

The architecture becomes much more concrete once the BANK and FUTUREBANK are taken seriously. The BANK begins with Γ. It then accumulates only material that has survived the relevant verification rule. In this sense, the BANK is the machine's verified mathematical past. The

FUTUREBANK is not a second theorem database. It is a representation of possible futures: what could happen if a particular deduction were tried, if a lemma were available, if the negation were pursued, or if attention were shifted from one role to another. This is no more "pie in the sky" than a chess engine considering hypothetical moves. The key is that a hypothetical move is not confused with a move that has actually been played, just as a hypothetical lemma is not confused with a theorem that has actually been verified.

This edition is deliberately written as a textbook rather than as a collection of research notes. It goes slowly. Each Part begins with a roadmap; each substantial chapter begins with a mini-roadmap. Artificial examples are used before realistic ones. Exercises appear throughout, and solution sketches are collected before the research frontier. The conjectures are moved to the end so that the reader first learns what the architecture *is* before being asked what it *might* eventually accomplish.

There is one additional intellectual influence worth naming carefully. Max Tegmark's 1998 paper on mathematical structures discusses "self-aware substructures" (SASs) and describes such a substructure as "something capable of some form of logical thought" [1]. Later he writes that "a substructure is self-aware if it thinks that it is self-aware" [1]. The formal self-awareness hierarchy used later in this book is *not* Tegmark's hierarchy; Tegmark did not define Levels $0, 1, 2, \omega$ for theorem provers. The influence is narrower: it motivates treating self-reference and formally represented self-description as legitimate structural subjects of mathematics. Here that idea is specialized to proof-search machines and separated sharply from claims about subjective consciousness.

The guiding sentence of the entire book is:

> **Imagine freely; verify rigidly.**

The BANK remembers what has actually survived. The FUTUREBANK represents what might happen. The COMPASS points. The controller plans. Formal self-description may reason about that process. The verifier alone decides what becomes mathematics.

Opening roadmap

Roadmap. Imagine the power of being able to tell whether a formal statement is true or false—in very informal terms. That sentence is deliberately intuitive rather than literally complete. An AMLD is not a universal truth oracle, and it cannot decide every formal statement. Relative to a declared background Γ and a fixed formal environment, its actual goal is to search for independently checkable evidence that a target c is provable, refutable, independent in the required metamathematical sense, or that the admitted background is inconsistent. If no such certificate is found under the declared conditions, the correct answer is UNKNOWN.

The power, if the architecture works well, comes from the breadth of statements that can be formalized. A target might be a theorem in pure mathematics, a software invariant, a protocol-safety property, an engineering reachability claim, a formally stated property of a scientific model, or a claim about whether an authentication system conforms to its specification. These application domains motivate AMLD research; none of them defines it.

Passwords and authentication appear later as *one worked application and one source of useful limiting examples*. They are not the central goal of AMLD, and the architecture is not being developed as a password-finding machine. Authentication is pedagogically valuable because it makes several general lessons vivid: search cannot manufacture information that is not present; a verifier may be stateful; external controls can sharply limit an otherwise powerful search procedure; and it can be more useful to search for a checkable defect in a formalized system than to search directly for a hidden secret. The same general architecture must ultimately be judged across mathematics and many other formal domains.

The book therefore proceeds from general to particular. First we define mathematical settlement and the certificate types sought by P, R, I, C. Next we build the shared BANK, the verifier boundary, state spaces, FUTUREBANK, imagination, planning, COMPASS control, and formalized self-description. Only after that foundation do we study applications—including authentication as one deliberately bounded special case—before turning to implementation, evaluation, exercises, and open conjectures.

Contents

Part I

What an AMLD Is

Chapter 1

From theorem proving to mathematical settlement

Roadmap. We begin with the simplest distinction in the book: proving one target is not the same task as settling its status. We define the five possible output statuses of an AMLD, explain why timeouts are not mathematical evidence, and identify exactly what the four search roles are trying to certify.

1.1 The ordinary ATP problem

An automated theorem prover, or ATP, is usually given a formal theory and a target formula. Its success condition is simple: find a proof. If the theory is represented by Γ and the target by c, the ATP searches for a certificate of

$$\Gamma \vdash c.$$

This is a perfectly coherent task. It is also one-sided. If the ATP does not find a proof, several very different explanations remain possible: perhaps c is false relative to Γ; perhaps c is independent of Γ; perhaps Γ is inconsistent; perhaps a proof exists but the heuristic missed it; perhaps the proof is merely too large for the current budget.

The point is not that theorem proving is defective. The point is that *non-success has many meanings.*

1.2 Settlement asks a larger question

An AMLD is designed around a larger question:

> What positive certificate, if any, settles the logical or metalogical status of c relative to Γ?

For this book, the principal roles are

$$\mathcal{A} = \{P, R, I, C\}.$$

Their intended terminal certificates are:

$$\begin{aligned} &P : \Gamma \vdash c, \\ &R : \Gamma \vdash \neg c, \\ &I : \text{a checked certificate that } \Gamma \nvdash c \text{ and } \Gamma \nvdash \neg c, \\ &C : \Gamma \vdash \bot. \end{aligned}$$

The letters name search roles, not truth values. P is a prover of c. R is a refuter of c, equivalently a prover of $\neg c$. I is a prover of an appropriate independence claim. C is a prover of contradiction.

These are *principal search objectives*, not ownership rules for all mathematics discovered during search. An agent may generate intermediate theorems, model facts, simplifications, or even a terminal certificate normally associated with another role. The BANK records the origin of such a discovery, but the verifier interprets the certificate according to its declared semantic kind.

1.3 Five honest statuses

A practical AMLD may return

PROVED, REFUTED, INDEPENDENT, INCONSISTENT, or UNKNOWN.

The first four are certificate-indexed conclusions. UNKNOWN is not a fifth mathematical truth status. It means that the machine has not produced a certificate licensing one of the other outcomes under the declared resource conditions.

> **Important distinction.** A timeout is a fact about a computation. Independence is a fact about derivability relative to a formal setting. The former does not become the latter merely because the timeout is long.

1.4 An artificial four-outcome example

Before discussing software, it helps to see tiny instances where each role has a natural job.

Example 1.1 (A P-win)**.** Let $\Gamma = \{p,\ p \to q\}$ and let $c = q$. The prover P may apply modus ponens to the two admitted premises. A verifier checks the inference and accepts q. The status is PROVED.

Example 1.2 (An R-win)**.** Let $\Gamma = \{\neg r\}$ and let $c = r$. The refuter already has a certified premise establishing $\neg c$. The status is REFUTED.

Example 1.3 (An I-win in propositional logic)**.** Let $\Gamma = \varnothing$ and let $c = p$. In classical propositional logic, take one valuation with p true and another with p false. The first is a model of $\Gamma \cup \{c\}$ and the second is a model of $\Gamma \cup \{\neg c\}$. By soundness, neither p nor $\neg p$ is derivable from the empty theory. A model-pair certificate can therefore establish INDEPENDENT relative to the declared propositional calculus.

Example 1.4 (A C-win)**.** Let $\Gamma = \{s, \neg s\}$ in ordinary classical logic. The contradiction agent can certify $\Gamma \vdash \bot$. The appropriate status is INCONSISTENT.

These examples are intentionally small. Their purpose is to separate the meanings of the four outcomes before search complexity enters the story.

1.5 Why contradiction is operationally special

In an explosive logic, contradiction is not just another symmetric victory.

Theorem 1.5 (Contradiction dominance under explosion)**.** *Assume the admitted logic validates explosion: from $\bot$ every formula is derivable. If C supplies a verified certificate of $\Gamma \vdash \bot$, then proofs of c and $\neg c$ cease to discriminate the intended status of c relative to that inconsistent background.*

Proof. By explosion, $\Gamma \vdash \bot$ implies $\Gamma \vdash c$ and $\Gamma \vdash \neg c$. Therefore the labels "proved" and "refuted" can both be obtained and no longer separate the two possibilities. The integrity status of the background has become the logically prior fact. □

A sensible explosive-logic implementation therefore halts with INCONSISTENT once an accepted contradiction certificate appears. In a paraconsistent logic the rule must be reconsidered; this is one reason the logical environment must be explicit.

Corollary 1.6 (Evidence overlap under inconsistency)**.** *Under the hypotheses of Theorem 1.5, if $\Gamma \vdash \bot$, then for every target c the three derivability conditions*

$$\Gamma \vdash c, \qquad \Gamma \vdash \neg c, \qquad \Gamma \vdash \bot$$

hold simultaneously. Thus the P, R, and C evidence conditions are not globally disjoint.

Proof. The contradiction condition is assumed. Explosion yields both c and $\neg c$. □

This is why the words *evidence condition* and *exclusive status* should not be silently identified. An accepted proof of c is a sound certificate of derivability even if the background is later discovered to be inconsistent. To interpret PROVED, REFUTED, INDEPENDENT, and INCONSISTENT as mutually exclusive logical classes, an additional consistency condition is needed.

Theorem 1.7 (Consistent-background status partition)**.** *Assume a classical metatheory and an object logic in which $\Gamma \vdash c$ together with $\Gamma \vdash \neg c$ entails $\Gamma \vdash \bot$. If Γ is consistent, meaning $\Gamma \nvdash \bot$, then exactly one of the following three metatheoretic conditions holds:*

(i) $\Gamma \vdash c$*;*

(ii) $\Gamma \vdash \neg c$*;*

(iii) $\Gamma \nvdash c$ *and* $\Gamma \nvdash \neg c$*.*

Moreover the contradiction condition $\Gamma \vdash \bot$ is false.

Proof. Classical metatheory gives the alternatives $\Gamma \vdash c$ or $\Gamma \nvdash c$, and independently $\Gamma \vdash \neg c$ or $\Gamma \nvdash \neg c$. Consistency rules out the case in which both derivability statements hold. The remaining possibilities are exactly (i), (ii), and (iii), and they are pairwise disjoint. By hypothesis $\Gamma \nvdash \bot$. □

Corollary 1.8 (Mutual exclusivity of sound terminal conclusions on a consistent background)**.** *Under the hypotheses of Theorem 1.7, if the terminal checkers are sound for their declared meanings, then accepted P-, R-, and I-type terminal certificates cannot correctly certify two different members of the partition for the same fixed* (Γ, c)*, and a sound C-certificate cannot exist.*

Corollary 1.9 (UNKNOWN is computational, not an additional logical alternative)**.** *Even when the hypotheses of Theorem 1.7 hold, an AMLD may return* UNKNOWN *under a finite resource budget. The theorem partitions the metatheoretic possibilities; it does not imply that the machine can always find a certificate for the true member of the partition.*

Exercise 1.10 (Status versus stopping)**.** For each event, classify it as computational stopping, mathematical settlement, both, or neither: (a) a 60-second timeout; (b) an accepted proof of c; (c) memory exhaustion; (d) a verified pair of models witnessing independence; (e) a search queue becomes empty without an exhaustive-closure certificate.

Chapter 2

The formal environment and the forgotten starting point: Γ

Mini-roadmap. This chapter introduces the object that must exist before there can be a BANK: the declared formal environment. We then make a crucial design decision: the BANK begins with explicit records for the starting axiom or hypothesis set Γ.

2.1 A certificate is never valid in a vacuum

To say that a proof is "valid" without saying relative to what is incomplete. We therefore fix a verification environment

$$\mathcal{E} = (\Gamma, c, \mathcal{L}, \mathcal{R}, \mathcal{C}_{\text{check}}, \nu),$$

where:

- Γ is the admitted axiom or hypothesis set for the instance;
- c is the target statement;
- $\mathcal{L}$ is the formal language;
- $\mathcal{R}$ is the declared inference system;
- $\mathcal{C}_{\text{check}}$ specifies certificate formats and their checkers;
- ν is a version identifier for the verification environment.

The version identifier is not decorative. If the checker, axiom base, parser, or admissibility rules change, the environment has changed.

2.2 Γ is the seed of mathematical memory

Earlier descriptions of the BANK can make it sound as though the bank begins empty and is later populated by discovered lemmas. That hides the most important first deposit. The machine has to begin from somewhere.

Definition 2.1 (Seed bank)**.** Let $\mathrm{seed}(\Gamma, \mathcal{E})$ be the collection of trusted initial records corresponding to the admitted axioms and hypotheses in Γ, with their scopes and environment identifier attached. The initial verified bank is

$$\mathsf{B}_0 = \mathrm{seed}(\Gamma, \mathcal{E}).$$

Thus the BANK starts with the formal assumptions of the problem. An axiom record is not being claimed to have been derived from earlier bank material. It is being recorded as an admitted starting premise of the environment.

> **Design principle.** The BANK should distinguish AXIOM/HYPOTHESIS records from DERIVED records. Both may be admissible premises, but their provenance is different.

2.3 Scope

Suppose a branch temporarily assumes λ in order to investigate "what follows if λ?" A statement proved under that assumption has scope $\Gamma \cup \{\lambda\}$. It may not be deposited as though it were proved from Γ alone.

A bank record should therefore include at least:

claim; certificate or admission tag; semantic kind; origin; scope; dependencies; provenance; verifier record; deposit time; environment ID.

This small amount of bureaucracy prevents large logical errors later.

Here *origin* and *semantic kind* are deliberately separate fields. Origin answers a historical question: "which search role submitted this record?" or "was this premise admitted by the environment?"

Semantic kind answers a logical question: is this an admitted hypothesis, an intermediate theorem, a target proof, a negated-target proof, an independence certificate, or a contradiction certificate?

2.4 A first walk through Γ and the BANK

Return to $\Gamma = \{p, p \to q\}$ and $c = q$. The machine begins with

$$\mathsf{B}_0 = \{[p : \mathsf{HYP}],\ [p \to q : \mathsf{HYP}]\}.$$

The prover reads both records, proposes the inference "modus ponens yields q," and submits its certificate. If V accepts it, the next state is

$$\mathsf{B}_1 = \mathsf{B}_0 \cup \{[q : \mathsf{DERIVED}]\}.$$

The bank did not merely store the answer. It stored the path by which the answer became legitimate.

2.5 A real formal theorem: $2 + 2 = 4$

The Metamath Proof Explorer contains the formally checked theorem `2p2e4`, whose assertion is

$$(2 + 2) = 4.$$

Its live theorem page displays the formal proof and dependencies [3]. This provides a concrete example of what an AMLD terminal event can look like.

Imagine an AMLD environment whose language and foundational assumptions are those of the relevant `set.mm` context, whose target is $c \equiv (2 + 2 = 4)$, and whose verifier is a compatible Metamath proof checker. The P role submits the existing proof certificate. The verifier checks it. If accepted, the AMLD returns PROVED.

This is a *real formally verified mathematical statement*, not merely the propositional toy example. At the same time, one should be precise about what is and is not being claimed: this paragraph describes a valid AMLD run at the architectural level using an existing Metamath certificate. It is not a claim that a particular current AMLD implementation independently rediscovered the published proof.

The example is pedagogically useful because it makes the boundary visible. Search may be trivial if the certificate is handed to P; verification is still meaningful. In harder instances the search problem becomes the dominant cost, but the acceptance semantics need not change.

Exercise 2.2 (Seeding the bank)**.** Let $\Gamma = \{a, a \to b, b \to d\}$ and target d. Write the initial bank records and a three-stage deposit history that ends with d. Which records are admitted premises and which are derived?

Chapter 3

The verifier: narrow on purpose

Mini-roadmap. We now isolate the verifier from the rest of the machine. The verifier should know enough to check a certificate and its admissible dependencies, but it should not care whether the compass likes the certificate or whether a reflective controller thinks the search is going well.

3.1 Role tags and semantic certificate kinds

Two classifications are needed, and they should not be conflated.

The *submitting role* is one of

$$\mathcal{A} = \{P, R, I, C\}.$$

Separately, let $\mathcal{T}$ be the set of declared semantic certificate kinds. It may include terminal kinds such as

TARGET_PROOF, NEGATED_TARGET_PROOF, INDEPENDENCE, CONTRADICTION,

as well as nonterminal kinds such as INTERMEDIATE_THEOREM, MODEL_FACT, or other environment-specific certificates.

For each semantic kind $t \in \mathcal{T}$, let $\mathcal{K}_t$ be its certificate-body space and define the semantically tagged certificate space

$$\mathcal{K}_{\text{sem}} = \bigsqcup_{t \in \mathcal{T}} (\{t\} \times \mathcal{K}_t).$$

A submitted proposal then lies in the role-tagged proposal space

$$\mathcal{K} = \bigsqcup_{a \in \mathcal{A}} (\{a\} \times \mathcal{K}_{\text{sem}}).$$

Thus a proposal has the conceptual form

$$k = (a, (t, \kappa)),$$

where a says *who submitted it*, t says *what kind of certificate it claims to be*, and κ is the certificate body.

Theorem 3.1 (Unique submitting-role tag)**.** *Every proposal $k \in \mathcal{K}$ has a unique submitting role $a \in \{P, R, I, C\}$ and a unique semantically tagged component $s \in \mathcal{K}_{\text{sem}}$ such that $k = (a, s)$.*

Proof. This is exactly the defining property of the tagged disjoint union. Every element belongs to one tagged component, and different role-tagged components are disjoint. □

Corollary 3.2 (No untyped agent proposal)**.** *If the AMLD selector is required to output elements of $\mathcal{K}$, every submitted proposal identifies exactly one of the four search roles. There is no fifth untagged agent proposal unless the architecture explicitly enlarges $\mathcal{A}$.*

The theorem is syntactic and architectural. It does *not* say that every theorem in the BANK is intrinsically a P-theorem, R-theorem, I-theorem, or C-theorem. A commutativity lemma found by R remains a commutativity lemma. Its origin tag records history; its semantic kind records what the verifier established.

Let

$$\tau : \mathcal{T}_{\text{terminal}} \longrightarrow \{P, R, I, C\}$$

map each terminal semantic kind to its designated settlement role: target proof to P, negated-target proof to R, independence to I, and contradiction to C. The designated settlement role need not equal the submitting role.

Corollary 3.3 (Terminal-role coverage)**.** *If the terminal semantic kinds are exactly the four kinds in the domain of τ, every accepted terminal certificate has a designated settlement role in $\{P, R, I, C\}$.*

Remark 3.4 (Submitting role versus settlement role)*.* Suppose R, while exploring a refutation, unexpectedly constructs a valid proof of c. The proposal may retain submitting-role tag R for provenance while carrying semantic kind TARGET_PROOF. If the checker accepts the certificate, the semantic settlement role is P and the derivability conclusion is PROVED. Nothing mathematically requires the discovery to be thrown away or relabeled as having been found by P.

Exercise 3.5 (Origin versus semantic kind)**.** An I-agent, while trying to build two models, discovers an ordinary object-level lemma L and later a valid proof of the target c. Give appropriate submitting-role tags and semantic kinds for both proposals. If both are accepted, what belongs in provenance, what belongs in theorem memory, and what terminal status does the second certificate license?

3.2 The intrinsic verifier

Let $\Gamma_{\mathcal{E}}(\mathsf{B})$ mean the base environment together with the verified bank records admissible in the certificate's scope. The intrinsic checker acts on semantic certificate content, not on the submitting-role tag. Define

$$v_{\mathcal{E}} : \mathfrak{B} \times \mathcal{K}_{\text{sem}} \longrightarrow \{0, 1\}.$$

A software wrapper may receive the full proposal $k = (a, s)$, record a as provenance, and pass only s together with its admissible dependencies to the intrinsic checker. The value 1 means that the submitted certificate is accepted. Real software may return richer diagnostics such as `REJECT`, `MALFORMED`, or `ERROR`; only acceptance authorizes a mathematical deposit.

Theorem 3.6 (Policy-neutral verification)**.** *Fix $\mathcal{E}$, a verified bank state B, and a semantic certificate $s \in \mathcal{K}_{\text{sem}}$. If the intrinsic verifier is defined only from $(\mathcal{E}, \mathsf{B}, s)$, then changing the Futurebank, search history, compass, controller, novelty score, formal self-model, or submitting-role tag cannot change the truth value of "s is accepted in B."*

Proof. Those coordinates are not arguments of the acceptance predicate. They may change which semantic certificate is selected, or who submits it, but not the verifier's value on a fixed input triple. □

This theorem is almost tautological. That is a feature. Good architecture often turns a difficult safety requirement into a simple type or interface fact.

Corollary 3.7 (Submitting-role neutrality)**.** *Let $a, b \in \mathcal{A}$ and let $s \in \mathcal{K}_{\text{sem}}$. If two proposals differ only in submitting-role tag, (a, s) versus (b, s), then their intrinsic verification result is identical in the same environment and admissible bank state.*

Proof. The intrinsic verifier receives the same semantic input s in both cases. The role tag is recorded as provenance but is not an argument of $v_{\mathcal{E}}$. □

Corollary 3.8 (Discovery-role irrelevance for terminal semantics)**.** *Under the same interface, if any search role submits a semantic certificate of kind* TARGET_PROOF *and the verifier accepts it, the mathematical conclusion is $\Gamma \vdash c$; analogously for the other terminal semantic kinds. Who discovered the certificate does not change what it proves.*

3.3 Independence is not dual timeout

The I role is the least like the other three. A proof of c, a proof of $\neg c$, and a proof of $\bot$ can all be object-level derivations. A proof of independence usually lives one level up.

In a model-theoretic setting, one sufficient pattern is a pair

$$M^+ \models \Gamma \cup \{c\}, \qquad M^- \models \Gamma \cup \{\neg c\}.$$

Under the appropriate soundness theorem, the first model rules out $\Gamma \vdash \neg c$ and the second rules out $\Gamma \vdash c$. In other settings, I may provide a formalized relative-consistency argument, a metatheoretic derivation, or an exhaustive finite certificate. The certificate type must be declared in $\mathcal{C}_{\text{check}}$.

The TPTP/SZS ecosystem contains a mature vocabulary for automated reasoning statuses, including semantic no-consequence style outcomes and model-based evidence [4, 5]. AMLDs should therefore be understood as part of a larger certificate-oriented automated reasoning tradition, not as if model-pair reasoning had no precedent.

3.4 The verifier as a boundary, not an oracle

A useful verifier is strict and narrow. It does not answer "is this statement really true in all possible senses?" It answers a declared formal question: does this certificate satisfy the checker for this environment and these dependencies?

That is enough to support a strong internal epistemology:

The machine may be uncertain about what to try next while being exact about what it has already certified.

Chapter 4

One AMLD cycle from start to finish

> **Mini-roadmap.** The pieces are now sufficient to describe a complete cycle: seed the BANK from Γ, choose a role, read verified material, propose an intermediate or terminal certificate, verify it, update state, and either halt or continue.

At step m, let the full state eventually have the form

$$\Sigma_m = (\mathsf{B}_m, \mathsf{F}_m, \mathsf{H}_m, \mathsf{Ref}_m, \mathsf{Ctrl}_m).$$

For the moment ignore all but B_m and H_m.

One turn is:

1. choose an active search role $a \in \{P, R, I, C\}$;
2. allow a to read any admissible verified records in B_m;
3. let a propose one new certificate or certified intermediate result;
4. submit it to V;
5. if accepted, deposit the resulting record in the appropriate typed compartment;
6. always record the attempt and diagnostics in verifier/search history;
7. if the accepted object is terminal, report the corresponding status and halt;
8. otherwise continue.

The distinction between *intermediate* and *terminal* accepted records is important. A bank can grow for a long time without the target being settled.

> **Checkpoint.** At this point the reader should be able to explain an AMLD without using the words Futurebank, compass, creativity, or self-awareness. Those ideas improve search. They are not required to define the basic correctness boundary.

Part II

The BANK: Verified Shared Memory

Chapter 5

Why a BANK changes the architecture

Roadmap. A bare conjecture settler can run several searches and wait for one to return a decisive certificate. The BANK makes the architecture cooperative and cumulative. This Part explains what is genuinely gained, what is not new in the broader literature, and which invariants make shared memory safe.

5.1 A race versus a community

Imagine four isolated processes. P searches for c, R for $\neg c$, I for independence, and C for contradiction. When any one succeeds, the system halts. That is a multiway conjecture settler, but the searches need not teach one another anything.

Now add a verified shared BANK. A lemma found while R is pursuing $\neg c$ may later help P. A model-theoretic construction found by I may expose a useful object-level lemma. A simplification found while C is testing consistency may help every role. The history of verified work becomes reusable.

The difference is not merely parallelism. It is *cross-role accumulation*.

Bare multiway settler	**BANK-centered AMLD**
Roles may search independently.	Roles read a common verified memory.
Intermediate work may disappear with a branch.	Accepted lemmas persist with provenance.
A role chiefly benefits itself.	A discovery by one role may help another.
The state is mainly "who is searching where?"	The state includes an accumulating mathematical object.
Search history and theorem memory may blur.	Verified BANK and verifier history are separated.

This use of shared memory has important precedents in blackboard architectures and cooperative theorem proving. For example, work on Leo-III explicitly explores an agent-based blackboard architecture [6]. The claim made here is therefore not that shared blackboards were invented

by AMLD. The architectural novelty *within this program* is the way a certificate-gated, typed, provenance-aware BANK is made central to four-way mathematical settlement and later paired with a quarantined Futurebank.

5.2 Shared does not mean untyped

The BANK is shared, but a metalogical independence statement is not automatically usable as an object-level theorem inside the theory whose independence is being discussed. A scoped hypothetical lemma is not automatically global. A contradiction certificate changes the interpretation of the whole run in explosive logics.

Hence the bank is both shared and typed.

5.3 Origin views record provenance, not ownership

The BANK itself is best treated as one set of verified records. Agent-specific "compartments" are useful indexes or views, but they should not define the ontology of the BANK because the initial Γ records were not discovered by any of P, R, I, C.

Let the origin-label set be

$$\mathcal{O} = \{\mathsf{ENV}, \mathsf{IMPORT}, P, R, I, C\}.$$

Here ENV marks axioms or hypotheses admitted directly by the current verification environment, while IMPORT may mark separately imported, already certified material. For $o \in \mathcal{O}$, define the origin view

$$\mathsf{B}_{o,m} = \{r \in \mathsf{B}_m : \mathrm{origin}(r) = o\}.$$

These views say where records came from. They do not say who owns them, and they do not determine their semantic kind.

Proposition 5.1 (Origin-view partition)**.** *If every BANK record has exactly one origin label in $\mathcal{O}$, then the origin views are pairwise disjoint and*

$$\mathsf{B}_m = \bigsqcup_{o \in \mathcal{O}} \mathsf{B}_{o,m}.$$

Proof. Every record has one origin, so it belongs to one origin view. Uniqueness of origin prevents membership in two distinct views. □

Corollary 5.2 (Seed records need no fictitious agent)**.** *Every initial record produced by* seed$(\Gamma, \mathcal{E})$ *may have origin ENV. Thus the theorem that every* agent proposal *has one of the tags P, R, I, C does not force the axioms of Γ to be attributed to an agent.*

> **Important distinction.** There are now two different classifications available for indexing. One may group records by *origin* (ENV, IMPORT, P, R, I, C), or by *semantic kind* (hypothesis, intermediate theorem, target proof, independence certificate, and so on). These classifications cross-cut each other. The same semantic kind may be discovered by different agents, and one agent may discover many semantic kinds.

5.4 Bank state versus bank state space

A state is one possible current bank. A state space is the collection of all bank states the machine could occupy. Let $\mathcal{R}_{\text{bank}}$ be the universe of well-formed BANK records for the fixed environment. A natural abstract state space is

$$\mathfrak{B} \subseteq \mathcal{P}_{\text{fin}}(\mathcal{R}_{\text{bank}}), \qquad \mathsf{B}_m \in \mathfrak{B},$$

where the subset restriction enforces the environment's typing, scope, provenance, and verification invariants. Origin views and semantic-kind indexes can then be derived from the record set rather than built into the mathematical identity of the BANK.

This notation is worth keeping clean because the full AMLD later has several nested state spaces.

Chapter 6

Provenance and monotonicity

Mini-roadmap. This chapter treats the BANK as a growing proof object. We show that accepted memory is monotone under the ordinary append-only design, and explain why provenance should survive even when two branches reach the same formula.

6.1 Append-only verified memory

The simplest BANK never deletes established records during a run. Search indices may be pruned, caches may be evicted, and representations may be compressed, but the logical content is monotonically accumulated.

Theorem 6.1 (Verified-bank monotonicity)**.** *Suppose the update rule only adds verifier-accepted records and never removes the logical availability of earlier accepted records. Then*

$$\mathsf{B}_m \subseteq \mathsf{B}_{m+1}$$

in the natural record-set interpretation for every m.

Proof. By the update rule, B_{m+1} is either B_m itself or B_m together with one or more newly accepted records. No old record is removed. □

This theorem is simple, but it has practical consequences. It allows dependency graphs, incremental indexing, replay, and audit trails to be defined against a stable growing object.

6.2 Provenance is part of mathematical memory

For every derived record r, store a dependency set $\mathrm{dep}(r)$. The dependency DAG can answer questions such as:

- Which axioms of Γ does this lemma ultimately use?
- Which role first produced the certificate?
- Which earlier bank records are needed to replay it?

- Does a supposedly global theorem actually depend on a temporary assumption?
- If a checker version is retired, which records must be revalidated?

The BANK can therefore be understood as a proof-carrying shared memory rather than a bag of formulas.

6.3 The BANK as realized history

Write $\mathsf{B}(0), \mathsf{B}(1), \ldots$ for snapshots through time. The sequence is itself a history. A ledger

$$L_{\mathsf{B}} \subseteq \text{RecordID} \times \mathbb{N} \times \mathcal{P}(\text{RecordID})$$

can record deposit times and dependencies.

This distinction will matter when we meet the FUTUREBANK. The BANK preserves the realized, verified past. The FUTUREBANK preserves represented possible futures.

Exercise 6.2 (Provenance error)**.** A branch assumes h temporarily, derives u from $\Gamma \cup \{h\}$, and then deposits u as though its scope were only Γ. Explain exactly what is wrong. Give one way a bank schema could make this error mechanically detectable.

Chapter 7

Verifier history is not the BANK

> **Mini-roadmap.** Rejected candidates can be useful data. The danger is allowing "useful" to drift into "proved." We therefore maintain a separate history H that search may learn from but deduction may not treat as theorem memory.

Let H_m contain events such as:

- accepted and rejected proposals;
- parser and type failures;
- duplicate candidates;
- timing and resource data;
- which Futurebank branch generated a proposal;
- compass scores at selection time;
- fallback invocations;
- controller actions.

A rejected lemma can lower the priority of similar candidates or trigger a structural escape. It cannot be used as a premise merely because it was interesting.

Theorem 7.1 (Verifier-gated noncontamination)**.** *Assume (i) B_0 contains only admitted seed records and validly certified imported records, (ii) every derived entry into B requires acceptance by a sound verifier for its declared environment, and (iii) no record in H or the Futurebank can bypass that gate. Then every finite bank state contains only admitted or validly certified records of the declared types and scopes.*

Proof. Induct on update steps. The base case is assumption (i). At the next step, rejected or merely speculative material changes no verified record. If a new derived record is added, assumption (ii) supplies its certification. Thus the invariant is preserved. □

> **Design principle.** Learning from failure is allowed. Reclassifying failure as proof is not.

Chapter 8

The shared lemma economy

Mini-roadmap. We now look at the BANK dynamically: agents withdraw by reading, deposit by verification, and sometimes produce lemmas whose value is discovered by a different role than the one that generated them.

8.1 Withdrawal does not consume

If P uses a bank theorem, the theorem remains available to R, I, C. The BANK is not a resource pile from which facts are removed. It is closer to a shared library with typed borrowing rights.

8.2 Cross-role value

Suppose R attempts to prove $\neg c$ and proves a general lemma L along the way. The verifier accepts L. Later P discovers that L is useful in a completely different derivation of c. This is not paradoxical. Intermediate mathematics need not inherit the final stance of the role that discovered it.

This is one reason agent names should be treated as provenance tags rather than epistemic tribes.

Corollary 8.1 (Cross-role reuse under origin-neutral admissibility)**.** *Suppose a verified BANK record r is admissible for a later inference exactly when its semantic kind, scope, dependencies, and environment satisfy the declared use rules, and suppose those rules do not restrict use merely because of* $\mathrm{origin}(r)$*. Then every search role may use r whenever those semantic admissibility conditions hold, regardless of which role originally submitted it.*

Proof. By hypothesis, admissibility is a function of the stated semantic conditions and not of origin. Changing only the role that originally discovered r therefore cannot change whether the record is admissible. □

8.3 When sharing can hurt

Shared memory can increase retrieval costs, create distraction, or flood the selector with irrelevant but valid lemmas. Correctness and efficiency must therefore be separated. A BANK may be logically

safe but computationally badly indexed.

Machine-learned relevance guidance can help rank large theorem libraries, and systems such as ENIGMA illustrate how learned ranking can guide symbolic theorem proving without making the learned model itself the final proof checker [7]. This is closely aligned with the AMLD principle that guidance may change search while verification remains independently defined.

Exercise 8.2 (Cross-role reuse)**.** Construct a toy example in which a lemma first derived by R is later useful to P. The lemma need not help R reach $\neg c$. Identify its scope and explain why agent origin does not restrict reuse.

Part III

Imagination: From BANK to FUTUREBANK

Chapter 9

Why chess is the right analogy

Roadmap. The word "imagination" can sound mystical when attached to a theorem prover. Chess removes the mystery. A chess engine considers moves that have not been played, follows their consequences, evaluates branches, and then chooses an action. We translate that idea into deductive search.

9.1 Chess search is counterfactual search

A chess engine at position x considers a legal move a, computes the successor $T(x, a)$, then considers replies from that position, and continues recursively. Claude Shannon's classic 1950 paper helped establish chess as a model problem for symbolic machine reasoning [2].

The engine's internal branch

$$x \to x_1 \to x_2 \to \cdots$$

does not mean those moves have occurred on the physical board. It means they are represented possibilities used for planning.

9.2 A deductive move

For theorem proving, a proof-search state x can include goals, local assumptions, substitutions, available bank records, and unresolved obligations. A *deductive move* is an action that changes this proof-search state by applying or hypothesizing a mathematically meaningful step.

Examples include:

- apply a rule of inference to selected premises;
- rewrite a goal using a bank theorem;
- introduce a quantified witness candidate;
- split into cases;
- introduce a temporary assumption for conditional reasoning;

- propose an auxiliary lemma;
- switch from object-level proving to model search;
- backtrack or change the active role.

The analogy is strong: a chess move changes a position; a deductive move changes a proof state. A sequence of moves creates a possible game; a sequence of deductive moves creates a possible proof history.

9.3 Where the analogy breaks

A chess engine can cheaply check whether many moves are legal. Mathematical proof search often contains a more expensive distinction. The system may want to explore

> "If lemma λ were available, would the target become easy?"

before investing the full cost required to prove λ. Thus some proof-search branches are conditional plans rather than fully certified deduction sequences.

This motivates a graded Futurebank.

9.4 The move/commit distinction

It is helpful to separate three verbs:

1. **Imagine**: represent a possible deductive move or hypothetical lemma.
2. **Attempt**: turn some imagined object into an actual certificate proposal.
3. **Commit**: after verifier acceptance, deposit the resulting record in the BANK.

A chess engine can imagine $Qh7+$ without playing it. An AMLD can imagine "prove λ, then rewrite by λ" without claiming λ is a theorem.

Exercise 9.1 (Chess-to-proof translation)**.** Give a chess-search concept and a proof-search analogue for each of the following: position, legal move, principal variation, transposition, evaluation function, and move actually played. Explain one place where the analogy is imperfect.

Chapter 10

The FUTUREBANK

Mini-roadmap. The FUTUREBANK is the provenance-preserving representation of possible proof futures. We define it first as a set of histories, then explain why a DAG is often a better implementation, and finally separate speculative from locally checked futures.

10.1 Histories, not just endpoints

Let $\mathrm{Succ}(x)$ be the represented successors of proof state x. If h is a history, let $\mathrm{end}(h)$ be its final state. Starting from the current history h_0, define

$$H_0 = \{h_0\},$$

and recursively let H_{d+1} contain one-step extensions of histories in H_d. The depth-d Futurebank is

$$\mathsf{F}_d = \bigcup_{j=0}^{d} H_j.$$

The emphasis on histories matters because two routes can reach the same visible endpoint while carrying different assumptions, dependency histories, or future options.

10.2 Tree versus DAG

A literal tree preserves provenance automatically but can duplicate enormous downstream computation. A DAG can merge equivalent future states, but only if the equivalence relation preserves everything relevant to future search and verification.

Therefore a Futurebank implementation should be willing to share computation while separately retaining whatever path multiplicity or ancestry the task needs.

10.3 A graded Futurebank

Distinguish

$$\mathsf{F}_{\mathrm{spec}}, \qquad \mathsf{F}_{\mathrm{loc}}, \qquad \mathsf{B}.$$

Interpret them as follows:

- $\mathsf{F}_{\mathrm{spec}}$: explicitly speculative branches; useful for planning, no theorem status;
- $\mathsf{F}_{\mathrm{loc}}$: branches that have passed a declared local admissibility or transition checker, but are not necessarily complete theorem certificates;
- B: material admitted or accepted by the final verifier.

The trust direction is

$$\mathsf{F}_{\mathrm{spec}} \longrightarrow \mathsf{F}_{\mathrm{loc}} \longrightarrow V \longrightarrow \mathsf{B}.$$

The arrows mean increasing certification, not increasing subjective confidence.

10.4 Futurebank does not mean truth

This is the defining quarantine rule. A node can have enormous compass score and still be speculative. A branch can be internally coherent and still depend on an unproved lemma. A controller can be nearly certain that a route will work and still be wrong.

Theorem 10.1 (Futurebank noncontamination)**.** *Under the hypotheses of Theorem 7.1, arbitrary additions to* $\mathsf{F}_{\mathrm{spec}}$ *cannot by themselves change the set of verified mathematical records in* B*.*

Proof. Speculative membership is not a deposit rule. Only verifier acceptance changes verified mathematical memory. □

Chapter 11

Planning, compass, and controlled creativity

Mini-roadmap. Imagination answers "what could happen?" Planning asks "what should we try?" The compass supplies ranking information, while a controller allocates finite attention. Creativity appears as controlled structural departure, not as permission to relax proof standards.

11.1 First-order compass

A first-order compass can score mathematical transitions:

$$K_1(x, y) \in \mathbb{R}.$$

High score means that moving from proof state x toward y appears promising under the learned or hand-designed navigation model.

The score may use features such as goal similarity, prior proof trajectories, quotient-state density, dependency structure, or resource estimates. None of these are truth predicates.

11.2 Second-order compass

Once the search itself is adaptive, a higher-level score can rank search actions:

$$K_2(\Sigma, a).$$

Actions might include:

- continue the current local branch;
- expand a different Futurebank region;
- switch from P to R or I;
- increase or decrease novelty pressure;

- backfill a speculative lemma;
- compress a quotient state;
- invoke complete fallback.

Thus K_1 points through mathematics while K_2 points through search policy.

11.3 Failure as information

Suppose K_1 repeatedly favors one region, but certificates generated there repeatedly fail verification. The disagreement between guidance and observed verification history is evidence about search quality. It can justify leaving the current neighborhood.

Controlled creativity can therefore be understood as a policy that increases willingness to explore structurally different futures when some combination of stagnation, novelty opportunity, and compass/failure tension crosses a threshold.

> **Design principle.** Creativity belongs in IMAGINE, COMPASS, SELECT, and controller state. It does not belong inside the verifier's acceptance rule.

11.4 Why imagination is not mystical

The phrase "the machine imagines a proof" should be read operationally:

> It recursively constructs and evaluates counterfactual deductive histories without confusing those histories with verified mathematics.

That is close in spirit to ordinary game-tree search. The mathematical challenge is not to make imagination metaphysical. It is to make counterfactual search useful, typed, provenance-preserving, and safe.

Chapter 12

Backfilling hypothetical lemmas

Mini-roadmap. A speculative branch can be valuable because it tells the machine what would be worth proving. We now show how a conditional plan can be converted into a verified route by proving its unmet dependencies in order.

Suppose a branch has the form

$$\text{if } \lambda_1, \lambda_2 \text{ are available, then } c \text{ becomes reachable.}$$

The Futurebank may represent this branch even when λ_1 and λ_2 are not in the BANK. The controller can then turn them into subgoals.

Proposition 12.1 (Dependency-closed promotion)**.** *Let a speculative branch depend on a finite acyclic set of hypothetical lemmas D. Suppose there exists an order $\lambda_1, \dots, \lambda_n$ of D such that each λ_j has a verifier-acceptable certificate from Γ together with earlier promoted dependencies $\lambda_1, \dots, \lambda_{j-1}$. Then the hypothetical dependencies can be promoted one by one to the BANK, after which the branch no longer depends on those hypotheses.*

Proof. Induct on the dependency order. The first certificate is valid from the admitted bank. After it is accepted and deposited, the second's prerequisites are available, and so on. Acyclicity ensures that the process has a topological order. □

This proposition does not say that every promising imagined branch can be backfilled. It says exactly what must be true for one to become real mathematics.

Exercise 12.2 (Backfill)**.** A speculative route to c requires lemmas λ and μ, where μ can be proved only after λ. Draw the dependency DAG. Describe the difference between (i) exploring consequences of both lemmas speculatively and (ii) promoting them into the BANK.

Part IV

Reflection and Formal Self-Description

Chapter 13

Tegmark's SAS idea and what this book borrows

Roadmap. This Part adds self-description without confusing it with imagination. We begin with Tegmark's notion of self-aware substructures as an intellectual influence, then define a syntactic reflection hierarchy for AMLDs. Finally we show that reflection may steer search while leaving proof semantics unchanged.

13.1 The Tegmark connection

In *Is "the theory of everything" merely the ultimate ensemble theory?*, Tegmark discusses mathematical structures complex enough to contain "self-aware substructures" and explicitly associates them with logical thought [1]. His project is cosmological and philosophical, not a design manual for automated theorem proving.

The relevance here is structural. A formal machine may contain a representation of itself and may reason about that representation. Once this possibility is admitted, one can ask precise questions: Which self-descriptions can it prove? How deeply can it iterate claims about its own provability? Which facts about its own search state can it safely use as controller input?

Important distinction. The Level-0, Level-1, Level-2, and Level-ω hierarchy below is a Tenneson-style formalization inspired by self-referential themes; it is not a hierarchy proposed by Tegmark. Nor is any level here a theorem about subjective consciousness.

13.2 A tagged machine

Let M be an AMLD with a formal name or code $\langle M \rangle$ available in an appropriate metatheory. A minimal self-description can have the form

$$\mathsf{AMLD}(\langle M \rangle).$$

The point is not that the string is magically profound. The point is that the system can formally refer to the machine as an object of reasoning.

13.3 A reflection sequence

For exposition, define a schematic reflection sequence

$$R_0(M) := \mathsf{AMLD}(\langle M \rangle),$$
$$R_{n+1}(M) := \mathrm{Prov}_M(\ulcorner R_n(M) \urcorner).$$

The exact provability predicate and coding must be supplied by the chosen metatheory.

A convenient hierarchy is then:

- **Level 0**: no nontrivial formal self-ascription of the designated kind;
- **Level 1**: the system can establish the designated base self-description $R_0(M)$;
- **Level 2**: it can establish the base description together with the next reflection statement $R_1(M)$;
- **Level** n: the relevant finite reflection prefix is available;
- **Level** ω: all finite levels are available in the intended sense of the framework.

This hierarchy measures formal self-description, not proof-search depth, IQ, consciousness, Futurebank size, or chess strength.

Chapter 14

Imagination and self-awareness are different axes

Mini-roadmap. A chess engine can search deep counterfactual trees without proving any sentence about itself. Conversely, a trivial formal machine can prove a coded statement about itself without possessing a useful Futurebank. We make this separation explicit.

Proposition 14.1 (Separation of basic imagination and basic self-description)**.** *Consider any class of computational search systems containing both (i) a system with nontrivial counterfactual/Futurebank search but no formal language in which it establishes the designated Level-1 self-description, and (ii) a tagged system that can establish the designated Level-1 self-description but has only a constant or nonbranching search policy. Within such a class, nontrivial Futurebank search does not imply Level-1 formal self-description, and Level-1 formal self-description does not imply nontrivial Futurebank search.*

Proof. System (i) is a counterexample to the first implication, and system (ii) is a counterexample to the converse. The claim is therefore a separation result about the two architectural properties, not a claim that either example has rich mathematical intelligence. □

The useful question is therefore not "does self-awareness create imagination?" It does not. The useful question is whether verified self-knowledge can improve the controller's use of imagination.

14.1 Futurebank-reflective AMLDs

Call an AMLD *Futurebank-reflective* relative to an architecture description if its formal language can name selected search components and it can establish correct statements about some of them, such as:

- "my current verified bank is B_m;"
- "branch f depends on speculative lemma λ;"
- "my controller has spent N verifier calls in this quotient region;"
- "these recent candidates were rejected;"

- “complete fallback has not received its scheduled share of steps.”

This is an additional coordinate of formal self-description. It should not be silently identified with the Level-n reflection hierarchy.

Chapter 15

Reflection can guide search but cannot redefine proof

> **Mini-roadmap.** We enlarge the full state to include a reflective coordinate. Then a one-line theorem shows why the verifier can remain immune to changes in reflection.

Let

$$\Sigma = (\mathsf{B}, \mathsf{F}, \mathsf{H}, \mathsf{Ref}, \mathsf{Ctrl}).$$

The selector may depend on all coordinates:

$$S : \mathfrak{S} \times \mathbb{N} \to \mathcal{K}.$$

The intrinsic verifier does not.

Theorem 15.1 (Reflection-neutrality of verification)**.** *Let two search states have the same environment $\mathcal{E}$ and admissible verified bank B but possibly different Futurebanks, histories, controllers, and reflective states. For any fixed semantic certificate $s \in \mathcal{K}_{\mathrm{sem}}$, the intrinsic verifier returns the same acceptance result in both states.*

Proof. This is Theorem 3.6 with the reflective coordinate made explicit. □

Corollary 15.2 (Reflective search without reflective relaxation)**.** *A reflective AMLD may use correct self-descriptive information to alter branch expansion, role allocation, creativity, backtracking, or fallback policy while preserving invariant proof semantics.*

The slogan is:

> Formal self-description may tell the machine something about how it is searching. It does not tell the verifier to lower the standard for proof.

Exercise 15.3 (Two axes)**.** Give four hypothetical systems occupying the four cells of a 2×2 table: low/high Futurebank capability crossed with low/high formal self-description. Explain why this table is more informative than a single "intelligence level."

Chapter 16

Safety, liveness, and fair fallback

Mini-roadmap. Verification gives a safety story: false or ill-scoped records should not enter the BANK. It does not by itself give a liveness story: the machine might search the wrong place forever. Fair scheduling and complete fallback address a different question.

16.1 Safety versus liveness

Safety asks: can an invalid candidate enter the verified bank or cause a false terminal status?

Liveness asks: if a discoverable certificate exists, will some component of the system eventually reach it?

A fixed sound verifier strongly supports safety. It does not ensure liveness because a selector can ignore the right proof forever.

16.2 Fair schedules

A schedule is fair to a component if that component receives infinitely many opportunities in an unbounded run. A simple round-robin schedule over P, R, I, C is fair. An adaptive scheduler can also be fair if it reserves nonzero recurring attention for each required process.

16.3 Complete fallback

Suppose there exists a baseline search F that is complete for a certificate class of interest. We may want a learned imaginative controller to dominate most of the computation while occasionally yielding to F.

Theorem 16.1 (Creative conservativity under fair complete fallback)**.** *Suppose a fallback procedure F is complete for finding any terminal certificate in a declared recursively searchable certificate class. Assume that F maintains persistent progress state and that each effective fallback step in the combined AMLD advances the same complete search that F would perform in isolation, or a monotone extension of it that never removes a candidate that the isolated search would eventually examine. If*

the AMLD schedule gives F infinitely many such effective steps, then arbitrary additional heuristic, imaginative, or reflective search interleaved with those steps does not destroy the completeness guarantee supplied by F for that certificate class.

Proof. If a certificate in the covered class exists, completeness of the isolated fallback search implies that it is encountered after some finite number N of effective fallback advances. By the persistence and coverage-preservation hypotheses, interleaving does not reset the fallback state or delete any candidate needed for those N advances. Fair interleaving eventually supplies at least N effective fallback steps. Other search may delay the calendar time of the Nth fallback step, but it cannot prevent the covered certificate from being reached. □

This theorem is conditional and modest. It does not make undecidable theories decidable. It says that imaginative search can be added conservatively around a complete procedure for whatever certificate class that procedure already covers.

Part V

Applications and Limits

Chapter 17

Unlimited hypernatural budgets: what they do and do not buy

Roadmap. Corollary 1.9 says that a finite-resource AMLD may honestly return UNKNOWN even when the underlying metatheoretic status of a standard sentence is already determined. Earlier versions of this research program also considered *unlimited hypernatural budgets*. This chapter explains exactly what changes in that idealization. The key result is strong: an unlimited hypernatural cutoff covers every standard finite PA proof. The key limitation is equally important: transfer is not a machine that turns an unlimited hypernatural or a hyperfinite nonfinding trace into an ordinary finite cutoff certificate.

17.1 The nonstandard setting

Fix an elementary nonstandard extension ${}^*\mathbb{N}$ of the standard natural-number structure $\mathbb{N}$, with the standard copy of $\mathbb{N}$ embedded in ${}^*\mathbb{N}$ and the corresponding transfer/elementarity principle available; for background on elementary extension and hyperfinite reasoning, see Keisler [14]. A hypernatural $H \in {}^*\mathbb{N}$ is called *unlimited* when

$$H > n \qquad \text{for every standard } n \in \mathbb{N}.$$

Merely writing $H \in {}^*\mathbb{N}$ is not enough: every ordinary natural is also represented among the hypernaturals. The extra hypothesis is unlimitedness.

Fix a standard Gödel coding of PA formulas and proofs. For a standard PA sentence φ, write

$$\mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner\varphi\urcorner),$$

for the standard decidable relation saying that p codes a valid PA proof whose conclusion has Gödel code $\ulcorner\varphi\urcorner$. To keep later formulas readable, we abbreviate this as $\mathrm{Prf}_{\mathrm{PA}}(p, \varphi)$. Its nonstandard extension is written ${}^*\mathrm{Prf}_{\mathrm{PA}}$.

For an unlimited H, let E_H denote the internal hyperfinite search that checks every hypernatural code

$$0 \le p \le H$$

against the transferred proof checker. This is not an ordinary finite computation viewed from outside the nonstandard universe. Internally, however, it is a bounded hyperfinite search.

The ordinary BANK state space introduced earlier consists of standard finite record sets. If one embeds the *entire AMLD state* into this nonstandard idealization and allows a run of hyperfinite length, its internal BANK and history belong to the corresponding nonstandard extensions and may be hyperfinite rather than standard finite. Equivalently, one may keep the argument here narrower and regard E_H as an auxiliary hyperfinite proof search whose result is interpreted externally. The two viewpoints should not be conflated.

> **Important distinction.** The words *standard sentence*, *standard natural*, and *unlimited hypernatural* are doing real work. In particular, the set of unlimited hypernaturals is an external object. The idealized decider below should therefore not be confused with an ordinary program executable for a merely very large standard number of steps.

17.2 Unlimited-bound proof equivalence

Theorem 17.1 (Unlimited-bound proof equivalence)**.** *Let $H \in {}^*\mathbb{N}$ be unlimited and let φ be a standard PA sentence. Then*

$$\mathrm{PA} \vdash \varphi \quad \Longleftrightarrow \quad (\exists p \in {}^*\mathbb{N})\big[p \le H \ \wedge \ {}^*\mathrm{Prf}_{\mathrm{PA}}(p, \varphi)\big].$$

The same equivalence holds with $\neg\varphi$ in place of φ, and with $\bot$ in place of φ.

Proof. If $\mathrm{PA} \vdash \varphi$, some ordinary finite proof has a standard Gödel code $n \in \mathbb{N}$. Unlimitedness gives $n < H$, so the hyperfinite search interval contains that same standard witness.

Conversely, suppose a hypernatural p satisfies ${}^*\mathrm{Prf}_{\mathrm{PA}}(p, \varphi)$. Then the transferred existential statement

$$(\exists p \in {}^*\mathbb{N})\, {}^*\mathrm{Prf}_{\mathrm{PA}}(p, \varphi)$$

holds. By elementarity/transfer for the standard proof predicate, the corresponding standard existential statement

$$(\exists n \in \mathbb{N})\ \mathrm{Prf}_{\mathrm{PA}}(n, \varphi)$$

holds. Hence $\mathrm{PA} \vdash \varphi$. The bound $p \le H$ is irrelevant for this reverse direction once a transferred proof witness exists. □

Corollary 17.2 (Unlimited-budget coverage of standard positive certificates)**.** *Every standard finite P-, R-, or C-certificate for PA lies below every unlimited hypernatural cutoff. Thus a hyperfinite exhaustive proof search through H cannot miss an existing standard PA proof of φ, $\neg\varphi$, or $\bot$.*

Corollary 17.3 (Standard positive-witness existence)**.** *If the hyperfinite search finds a possibly nonstandard witness p satisfying the transferred PA proof predicate for a standard sentence φ, then an ordinary finite PA proof of φ exists. The particular hypernatural witness p need not itself be standard.*

The last sentence matters. The conclusion is that a standard proof *exists*; it is not an algorithm for extracting a standard proof directly from the particular nonstandard witness. Once existence is known metatheoretically, an ordinary complete enumeration of finite PA proofs will eventually find some standard witness, although there is no claim here that the hypernatural witness itself supplies an effective shortcut to it.

17.3 An idealized hypernatural decider

Assume for this section that PA is consistent. Define an external classification rule D_H on standard PA sentences by inspecting the completed hyperfinite searches for φ and $\neg\varphi$:

$$D_H(\varphi) = \begin{cases} P, & \text{if a proof of } \varphi \text{ occurs by } H, \\ R, & \text{if a proof of } \neg\varphi \text{ occurs by } H, \\ I, & \text{if neither occurs by } H. \end{cases}$$

If inconsistency is not assumed away, a C-test for a proof of $\bot$ should receive the contradiction-dominance priority described in Theorem 1.5.

Theorem 17.4 (Hyperfinite trichotomy for standard PA sentences)**.** *Let PA be consistent, let H be unlimited, and let φ be a standard PA sentence. Exactly one of the following holds:*

(i) *E_H finds a proof of φ;*

(ii) *E_H finds a proof of $\neg\varphi$;*

(iii) *E_H finds neither.*

Moreover these three cases correspond respectively to

$$\mathrm{PA} \vdash \varphi, \qquad \mathrm{PA} \vdash \neg\varphi, \qquad \mathrm{PA} \nvdash \varphi \ \text{ and } \ \mathrm{PA} \nvdash \neg\varphi.$$

Proof. The correspondence in the first two cases is Theorem 17.1. If neither hyperfinite search finds a proof, then by the same equivalence neither standard derivability statement holds. Conversely, if neither standard derivability statement holds, transfer of the two universal non-proof statements says that there is no transferred proof witness anywhere in ${}^*\mathbb{N}$, hence certainly none below H. Consistency excludes the possibility that both first and second cases occur. □

Corollary 17.5 (Idealized external settlement)**.** *Under the hypotheses of Theorem 17.4, D_H classifies every standard PA sentence into the correct P, R, or I metatheoretic case. If a C-search is included and PA is inconsistent, every standard proof of contradiction is likewise covered below H.*

This is a genuine strengthening of the finite-budget picture, but it is an *idealized nonstandard* strengthening. It does not contradict undecidability of ordinary computable PA theoremhood, because the completed search uses a nonstandard amount of internal time and the externally stated hypothesis that H is unlimited.

17.4 Why no ordinary finite cutoff can replace an unlimited one

Theorem 17.6 (Finite-cutoff insufficiency)**.** *For every standard $N \in \mathbb{N}$, there exists a standard PA theorem θ for which no proof code of θ is at most N.*

Proof. There are only finitely many natural numbers $p \le N$, hence only finitely many proof objects with codes at most N, and therefore only finitely many theorem conclusions represented by such proofs. PA has infinitely many distinct theorems; for example, the reflexive equalities for distinct numerals give infinitely many standard theorem statements. Consequently at least one PA theorem has no proof whose code is at most N. □

Corollary 17.7 (No universal standard exhaustion bound)**.** *Failure to find a proof below one fixed standard finite cutoff N cannot, by itself, establish nonderivability for arbitrary PA targets. No standard N has the cofinal coverage property of an unlimited H.*

This theorem makes precise why "replace H by a gigantic ordinary integer" does not preserve the argument. However large the ordinary integer is, some ordinary finite PA proofs lie beyond it.

17.5 The direct-standardization barrier

Proposition 17.8 (Direct-cutoff standardization barrier)**.** *Let $H \in {}^*\mathbb{N}$ be unlimited. There is no standard natural N that preserves the defining cutoff property*

$$\textit{every standard } n \in \mathbb{N} \textit{ satisfies } n \leq H$$

by simply replacing H with N. Consequently a hyperfinite negative-search record such as

$$(\forall p \leq H)\, \neg {}^*\mathrm{Prf}_{\mathrm{PA}}(p, \varphi)$$

cannot be converted into an equally exhaustive ordinary finite-search record merely by taking a standard natural replacement for H.

Proof. If a standard N dominated every standard natural, then $N+1$ would be a standard natural with $N+1 > N$, a contradiction. Thus no standard cutoff has the same cofinal coverage property. The conclusion about the search record follows because replacing H by any standard N discards standard proof codes larger than N. □

> **Important distinction.** Transfer is not an elementwise "nonstandard-to-standard converter." In particular, the usual standard-part construction applies to finite hyperreals; an unlimited hypernatural has no natural-valued standard part that preserves its cutoff property. For the P, R, and C cases, Theorem 17.1 gives existence of an ordinary finite witness. For the I case, the hyperfinite nonfinding trace plus the external fact that H is unlimited gives an external independence argument, but the trace itself is not automatically an ordinary finite independence certificate in the AMLD's declared $\mathcal{K}_I$ format.

There is an additional nuance. In formal frameworks where a nonstandard theory is known to be conservative over an appropriate standard metatheory, a fully formalized nonstandard *argument* may sometimes be translated into a standard metaproof. That is a separate proof-theoretic claim. It should not be confused with taking the particular unlimited cutoff H or its hyperfinite trace and mechanically turning that object into a standard finite cutoff.

Corollary 17.9 (Reconciliation with computational UNKNOWN)**.** *Corollary 1.9 and Theorem 17.4 are compatible. An ordinary AMLD with a standard finite budget may return* UNKNOWN*. An idealized nonstandard AMLD supplied with an unlimited hypernatural budget can externally classify every standard PA sentence by exhaustive proof search under the hypotheses above. What does not follow is that every such hypernatural I-settlement is already packaged as an ordinary finite I-certificate.*

Exercise 17.10 (Unlimited versus merely enormous)**.** Let $H \in {}^*\mathbb{N}$ and let $N \in \mathbb{N}$ be an extremely large standard integer.

(a) Why does $H \in {}^*\mathbb{N}$ alone not imply that H covers every standard proof code?

(b) If H is unlimited and PA proves a standard sentence φ, why must E_H encounter a proof of φ?

(c) Why can no fixed standard N replace H in that universal coverage statement?

(d) Explain the asymmetry between finding a $P/R/C$ witness and obtaining an I-classification from hyperfinite nonfinding.

Chapter 18

Authentication as one worked special case

Roadmap. This chapter is an application, not the destination of the theory. Passwords and authentication are used because they sharply separate *search*, *verification*, *available information*, and *system design*. We will model a password verifier, define "flood control" as a hard or effective bound on online attempts, prove several finite-attempt theorems, distinguish online from offline verification, and show why a more plausible defensive use of AMLD-style reasoning is often to settle formal claims about an authentication system rather than to "guess the password." The same AMLD architecture is intended for much broader application classes, including ordinary mathematical settlement, software and protocol verification, engineering safety, and formalized scientific reasoning. The discussion here is about authorized analysis, defensive design, and formal modeling.

18.1 Why passwords belong in this book

A password system looks superficially like proof search. A candidate string is proposed; a verifier returns an answer; a search policy decides what to try next. That resemblance is real, but it must not be exaggerated.

A mathematical verifier answers a question such as "is this certificate a valid proof in environment $\mathcal{E}$?" A live authentication verifier is usually *stateful*: it may count failures, insert delays, require an additional factor, reject an otherwise correct credential after a lockout, or stop accepting attempts altogether. For that reason, password authentication is best treated as a *special case of guarded search with verification*, not as literal mathematical theorem proving.

This chapter also illustrates a broader policy point. Powerful search machinery should be developed together with narrow correctness boundaries and explicit use constraints. In the companion essay *Fund the Proof Machines—Guard the Capabilities That Can Become Weapons*, Tenneson argues that automated theorem proving differs from human-mimicking AI because its distinctive power is the search for independently checkable chains of reasoning. The essay treats passwords as only one dual-use example alongside mathematical discovery, software verification, protocol analysis, engineering safety, cryptographic analysis, and other formally stated targets [10]. That is also the role passwords play here: they are a revealing example, not the organizing purpose of AMLD.

Important distinction. Nothing in the AMLD architecture acts like a magic eight ball that can pluck a secret password from the sky. If a password is independent of everything the machine can observe, then imagination, reflection, a larger FUTUREBANK, or a more elaborate COMPASS cannot manufacture information about that secret. They can only change the order in which candidates are considered. Theorems below make this precise.

18.2 A minimal authentication model

Let S be a finite set of candidate secrets and let the actual secret be a random variable $W \in S$. Let Z denote all side information legitimately available to the analysis. Conditional on Z, write

$$\pi(w) = \Pr(W = w \mid Z).$$

The distribution π is the search prior. A truly random, independently generated secret makes π close to flat over its effective support. Human choice, reuse, policy constraints, previously disclosed information, or implementation artifacts may make π highly nonuniform.

For a live system, let

$$\mathsf{Auth}_t(g, x_t) \in \{\mathsf{ACCEPT}, \mathsf{REJECT}, \mathsf{DELAY}, \mathsf{BLOCKED}\}$$

be the authentication response to candidate g in system state x_t. The state may include a retry counter, lockout timer, risk score, second-factor state, account state, and other policy variables.

Definition 18.1 (Flood control)**.** In this book, *flood control* is an umbrella term for controls that bound or slow repeated authentication attempts. Standard security terminology includes *rate limiting*, *login throttling*, and *account lockout*. If the system will process at most N candidate submissions before refusing or materially delaying further attempts, we call N the *online attempt budget* for the modeled episode.

NIST requires effective rate limiting for password authentication, and its current digital-identity guidance gives an upper bound of 100 consecutive failed attempts for authenticator types covered by the general rule, while allowing lower limits and additional delays or bot-mitigation controls [11]. OWASP likewise recommends login throttling and discusses lockout thresholds, observation windows, lockout duration, and the denial-of-service risk of overly rigid lockout policies [12].

18.3 The right quantity is top-N probability mass

Suppose the candidates are ordered so that

$$\pi_{(1)} \geq \pi_{(2)} \geq \cdots \geq \pi_{(M)}, \qquad M = |S|.$$

Define the *top-N exposure*

$$G_N(\pi) = \sum_{i=1}^{\min(N,M)} \pi_{(i)}.$$

This quantity answers a concrete question: if only N online guesses are possible, what is the greatest success probability available to a strategy that receives no extra information except ordinary accept/reject responses?

Theorem 18.2 (Optimal finite-attempt guessing bound)**.** *Assume a strategy may submit at most N distinct candidates, receives no side channel or new information about W other than whether a submitted candidate equals the secret, and cannot bypass the attempt budget. Then its maximum probability of success is exactly*

$$G_N(\pi).$$

An optimal strategy tries candidates in nonincreasing order of π.

Proof. Any strategy that makes at most N distinct guesses succeeds only when W lies in the set of guessed candidates. The probability mass of any such set is at most the sum of the N largest point probabilities, namely $G_N(\pi)$. Guessing those N candidates attains that bound. Rejections merely remove already-tried candidates; under the stated assumptions they reveal no additional information that changes the relative ordering of the untried prior masses. □

Corollary 18.3 (Uniform-secret flood-control bound)**.** *If W is uniform on a set of size M, then any such N-attempt online strategy succeeds with probability at most*

$$\min\left(1, \frac{N}{M}\right).$$

This is a clean mathematical form of the motivating intuition that flood control can make an otherwise powerful search method nearly irrelevant to direct online password guessing. If $N \ll M$ and the prior is close to uniform, there simply is not enough interaction budget for sophisticated search to matter much.

18.4 No-information imagination cannot create information

The AMLD vocabulary lets us say the same thing more structurally.

Theorem 18.4 (No-oracle theorem for password imagination)**.** *Under the hypotheses of Theorem 18.2, adding a FUTUREBANK, a COMPASS, self-description, multiple search agents, or arbitrary internal computation cannot raise the success probability above $G_N(\pi)$ unless the added mechanism has access to information that changes the conditional distribution of W or changes the verifier/attempt constraints.*

Proof. Internal computation can choose an ordering or a set of at most N candidates, but success still implies that W lies in that set. Theorem 18.2 bounds the probability mass of every such set. To exceed the bound, some hypothesis must change: new information must alter π, the verifier must leak information, the candidate equivalence relation must differ from the model, or the attempt policy must be bypassed or altered. □

> **Design principle.** For passwords, imagination is therefore best understood as *hypothesis generation about structure*: candidate distributions, verifier behavior, recovery routes, storage assumptions, policy inconsistencies, or specification defects. It is not a source of secret bits.

18.5 A min-entropy version of the flood-control theorem

Let

$$H_\infty(W \mid Z) = -\log_2 \left(\max_w \pi(w) \right)$$

be the conditional min-entropy.

Theorem 18.5 (Flood-control min-entropy bound)**.** *Under the hypotheses of Theorem 18.2,*

$$\Pr(\textit{success in at most } N \textit{ guesses}) \le \min\left(1, N\, 2^{-H_\infty(W|Z)}\right).$$

Proof. Every point probability is at most $2^{-H_\infty(W|Z)}$. The probability mass of any N guessed candidates is therefore at most N times that quantity, capped by 1. □

This theorem explains why a small N is powerful only when it is combined with a password distribution whose largest point probabilities are themselves small. The nominal size of the character alphabet is not enough. What matters to a capped online attacker is how much probability mass lies in the first few guesses.

18.6 Human-chosen passwords, random passwords, and password managers

The top-N theorem turns password quality into a distributional statement.

- A unique randomly generated password aims to flatten π, making $G_N(\pi)$ small for realistic N.
- Reuse or highly predictable human choice concentrates π, making a small set of candidates carry more probability mass.
- Password blocklists remove especially high-probability choices from the admissible set. NIST explicitly connects blocklists to the fact that online attempts are rate-limited [11].
- Password managers make long, unique, machine-generated secrets practical and reduce the human incentive to reuse memorable strings.

Thus "strong password" should not be reduced to a composition-rule slogan. In the finite-attempt model, the central operational quantity is $G_N(\pi)$ or a bound on it.

18.7 Online and offline verification are different worlds

Flood control belongs to the *live verifier*. It may be decisive online and irrelevant offline.

Proposition 18.6 (Online/offline separation)**.** *Suppose an online service enforces an attempt budget N, but a compromise exposes data sufficient to test password guesses locally without querying that service. Then the online attempt cap does not bound the number of local verification trials. The defensive cost per offline trial and the password distribution become separate controlling factors.*

Proof. The online cap is enforced by the live service state. A local verifier that does not consult that state does not consume the service's retry counter. Therefore the online bound does not apply to those local evaluations. □

This is why password storage matters. OWASP recommends one-way password hashing rather than reversible encryption, unique salts, and deliberately expensive password-hashing functions so that each offline candidate test has nontrivial cost [13].

Theorem 18.7 (Salt partition principle)**.** *Model a stored verifier as $y = F(s, w)$ where s is an account-specific salt. If two accounts use different salts $s_1 \neq s_2$, then a table of values precomputed only for inputs of the form $F(s_1, \cdot)$ is not, by itself, a table of verifier values for $F(s_2, \cdot)$. Verification work must be parameterized by the second salt as well.*

Proof. The salt is an explicit input to F. Values computed at (s_1, w) are values of a different input family than (s_2, w). Unless additional structure of F collapses the two families, the first table does not directly supply the second family of outputs. □

The theorem is intentionally modest. It does not claim that salting makes weak passwords safe. It states the narrower reason salts defeat straightforward cross-account reuse of a single precomputed password-to-hash table.

18.8 Multi-factor authentication changes the target predicate

A modern authentication decision may not be "password correct?" It may be

$$\mathsf{PasswordOK} \wedge \mathsf{SecondFactorOK} \wedge \mathsf{AccountEligible} \wedge \mathsf{PolicyAllows}.$$

Theorem 18.8 (Conjunctive-factor containment)**.** *If a system accepts only when both password condition P and an additional factor condition F hold, then the set of fully accepted authentication events is a subset of the events satisfying P alone. Consequently, success at the password subproblem is not sufficient for full authentication.*

Proof. Every event satisfying $P \wedge F$ satisfies P. The converse need not hold. □

OWASP describes multi-factor authentication as one of the strongest defenses against password-related attacks [12]. In AMLD terms, this means that the terminal certificate predicate has changed: a password candidate is no longer the whole certificate.

18.9 The more interesting target: prove the verifier conforms to its specification

Direct secret guessing is often the least interesting formal problem. A much richer defensive target is a claim about the authentication system itself.

Let $\mathsf{Req}(z)$ be the *required/intended* acceptance predicate for a complete authentication event z, and let $\mathsf{Impl}(z)$ be the implemented acceptance predicate. Consider the conformance claim

$$c_{\text{auth}}: \qquad \forall z\,[\mathsf{Impl}(z) \Rightarrow \mathsf{Req}(z)].$$

This says: every event the implementation accepts is an event the security specification intends to accept. The notation deliberately avoids the letters I and A, which already have architectural meanings elsewhere in the book.

Theorem 18.9 (Authentication-defect certificate)**.** *Assume that, for a concrete event z^*, the declared audit checker can verify both $\mathsf{Impl}(z^*) = \mathit{true}$ and $\mathsf{Req}(z^*) = \mathit{false}$ from finite representations. Then z^* together with those checker receipts is a finite refutation certificate for*

$$c_{\text{auth}}: \quad \forall z\,[\mathsf{Impl}(z) \Rightarrow \mathsf{Req}(z)].$$

Proof. The verified witness z^* is a counterexample to the universal implication $\forall z[\mathsf{Impl}(z) \Rightarrow \mathsf{Req}(z)]$. The finite checker receipts make the counterexample independently checkable in the declared audit environment. □

This is the sense in which AMLD-like reasoning may be more useful for passwords: not as a machine that guesses secrets, but as a machine that searches for *certifiable flaws in the modeled authentication system*. Examples of defensively relevant specification surfaces include retry policies, recovery flows, inconsistent enforcement of second factors, account-state transitions, password-storage configuration, overly informative error behavior, parser/normalization consistency, and mismatches between documented policy and implemented policy. OWASP specifically warns that authentication error behavior can disclose account-existence information and recommends generic responses [12].

> **Important distinction.** A defect witness is not a recovered password. The two are different mathematical objects. The useful security question is often "does the implementation admit any event forbidden by the specification?" rather than "what is the secret?"

18.10 A four-role AMLD audit of an authentication system

The conformance target gives the four AMLD roles concrete defensive meanings.

- P : prove c_{auth} for the modeled state space;
- R : produce a checked counterexample event z^*;
- I : certify that the available formal specification is insufficient to decide c_{auth};
- C : derive an inconsistency in the declared security specification or model assumptions.

The BANK begins with the formal authentication specification, the modeled implementation semantics, and explicitly admitted environmental assumptions. Verified invariants and checked test lemmas can be deposited as the audit proceeds. The FUTUREBANK may hold possible counterexample traces or repair hypotheses, but none becomes an established defect or theorem without a checker-accepted certificate.

This is a much better fit to the book's architecture than pretending that a hidden password is a theorem waiting to be divined.

18.11 Recovery routes and the weakest-route upper bound

Authentication rarely has only one route. There may be password login, password reset, account recovery, delegated identity, support-assisted recovery, remembered sessions, or hardware-backed authentication.

Theorem 18.10 (Weakest-route upper bound)**.** *Suppose any one of routes $R_1, \ldots, R_k$ is sufficient to obtain the same protected account capability, and let C_i be the minimum cost of successfully traversing route R_i under a fixed threat model. Then the minimum cost of obtaining the capability satisfies*

$$C_{\text{system}} \leq \min_i C_i.$$

Proof. An actor may choose the least-cost sufficient route. Therefore system-level cost cannot exceed the cost of the easiest sufficient route. □

The theorem is deliberately an upper bound, not a complete security metric. Its lesson is architectural: strengthening the password path while leaving a weaker recovery path unchanged may not strengthen the whole account by the same amount.

18.12 Flood control can itself create a control problem

A hard lockout reduces online guessing but may also make availability vulnerable if arbitrary outsiders can consume the failure budget of someone else's account. OWASP explicitly notes this denial-of-service tradeoff and discusses alternatives such as delays and graduated controls [12].

This creates an AMLD-style multiobjective control problem. The controller is not optimizing only

$$\text{minimize unauthorized acceptance.}$$

It is balancing at least

unauthorized acceptance risk,	legitimate-user lockout risk,	
latency,	recovery burden,	operational cost.

There is no universal theorem that one fixed N is optimal for every system. The correct N, delay schedule, and recovery policy depend on the threat model and operational environment.

18.13 The stateful-verifier mismatch with mathematical proof

Proposition 18.11 (Authentication verification is generally nonmonotone in time)**.** *There exist ordinary authentication policies for which the same credential submission is accepted in one system state and blocked in another, even though the credential string itself has not changed.*

Proof. A lockout or disabled-account state provides an immediate example: the underlying password may be correct while policy blocks authentication in the later state. □

This is unlike the intended verifier semantics of the BANK. Once a proof certificate is valid relative to a fixed environment version, later search history should not make that same certificate invalid. The password analogy therefore teaches an important boundary: search architecture may transfer across domains even when verification semantics do not.

18.14 What an AMLD can realistically contribute to password security

A disciplined research program would emphasize the following defensive tasks:

- formalize the intended authentication policy and prove invariants about it;
- search for finite counterexample traces to policy conformance;
- verify that flood-control rules actually impose the modeled attempt budget;
- check that alternate login and recovery routes satisfy the same intended security requirements;
- reason about the distributional quantity $G_N(\pi)$ rather than relying on nominal password-space size alone;
- compare fixed lockout, delay, and adaptive throttling policies under explicit availability constraints;
- verify that password storage follows a declared modern hashing-and-salting policy;
- test whether error channels reveal facts that the formal policy intends to conceal;
- preserve failed audit hypotheses in verifier history without promoting them into the BANK as defects;
- use the FUTUREBANK for hypothetical repairs, then require regression certificates before accepting a repair as safe.

The likely payoff is therefore *finding and certifying flaws in a system or certifying that a modeled class of flaws is absent*, not mysteriously recovering an otherwise information-theoretically hidden password.

18.15 Worked numerical example: when N makes guessing negligible

Suppose a password is uniformly random over $M = 2^{80}$ possibilities and the live verifier will process at most $N = 100$ candidate submissions in the modeled episode. Corollary 18.3 gives

$$\Pr(\text{success}) \leq \frac{100}{2^{80}} \approx 8.27 \times 10^{-23}.$$

No clever ordering changes this because all candidates have equal prior probability. A COMPASS may be computationally impressive and still be informationally irrelevant.

Now change only the prior. If human behavior or leaked side information concentrates, say, 5% of the conditional probability mass into the first 100 candidates, then $G_{100}(\pi) = 0.05$ and the same

flood-control value $N = 100$ no longer gives the same protection. The difference is not "more imagination." It is a different information model.

18.16 Exercises

Exercise 18.12 (Top-N exposure)**.** Let $S = \{a, b, c, d, e\}$ with probabilities

$$(0.40, 0.25, 0.15, 0.10, 0.10).$$

Compute G_1, G_2, and G_3. Then explain why, with no new side information, a more elaborate search architecture cannot exceed those success probabilities under attempt budgets $1, 2, 3$.

Exercise 18.13 (Flood control versus offline verification)**.** Explain why a server-side $N = 10$ lockout can strongly constrain online guessing while placing no direct ten-trial limit on a hypothetical local verifier obtained from compromised password-verifier data. Name the separate defense that becomes important in the local-verification setting.

Exercise 18.14 (Turn security into a settlement problem)**.** Write a conformance statement c for a toy authentication system in which acceptance is intended to require both a password predicate $P(z)$ and second-factor predicate $F(z)$. What kind of object would let R refute c?

Exercise 18.15 (Flood-control tradeoff)**.** Give a toy example in which lowering the maximum failure count decreases online guessing opportunity but increases the possibility that a legitimate user is denied service. Explain why this is a control-design problem rather than a theorem that "smaller N is always better."

Part VI

Implementation

Chapter 19

Data structures

Roadmap. The mathematics now becomes software. This Part gives pseudocode for the AMLD state, the narrow verifier interface, Futurebank generation, role scheduling, backfilling, safe quotienting, and invariant tests.

Listing 19.1: Core records and state

```
record VerificationEnvironment:
    gamma_records        # admitted axioms/hypotheses Gamma
    base_scope           # logical scope of the seed assumptions
    target               # c
    language_id
    inference_system_id
    checker_versions
    environment_id

record BankRecord:
    claim
    semantic_kind        # AXIOM, HYPOTHESIS, INTERMEDIATE_THEOREM,
                         # TARGET_PROOF, INDEPENDENCE, other declared kinds
    origin               # ENV, IMPORT, P, R, I, or C
    scope
    certificate
    dependency_ids
    verifier_receipt
    deposited_at
    environment_id

record Proposal:
    submitting_role      # exactly one of P, R, I, C
    semantic_kind        # what the certificate claims to establish
    certificate_type     # selects the formal checker
    certificate
    scope
    dependency_ids
    environment_id

record LogicalBank:
    records_by_id
    origin_index         # ENV, IMPORT, P, R, I, C -> record IDs
    semantic_kind_index  # semantic kind -> record IDs
    dependency_graph
```

```
record FutureNode:
    proof_state
    parent_ids
    generating_action
    role
    assumed_dependencies
    bank_dependencies
    depth
    trust_grade          # SPEC or LOCAL
    compass_score
    estimated_cost
    attempted = false

record SearchState:
    bank
    futurebank
    history
    reflection_state
    controller_state
    step
```

19.1 Seeding from Γ

Listing 19.2: The BANK starts from Gamma

```
function SEED_BANK(env):
    B = empty LogicalBank
    for premise in env.gamma_records:
        r = BankRecord(
            claim = premise.formula,
            semantic_kind = premise.kind, # AXIOM or HYPOTHESIS
            origin = ENV,
            scope = premise.scope or env.base_scope,
            certificate = premise.admission_record,
            dependency_ids = {},
            verifier_receipt = "ADMITTED-BY-ENVIRONMENT",
            deposited_at = 0,
            environment_id = env.environment_id
        )
        B.add(r)
    return B
```

This function makes the conceptual starting point explicit: Γ is not hidden in global code. It becomes auditable initial mathematical state. The seed records use origin ENV, not a fictitious P, R, I, or C origin. Imported certified libraries can use IMPORT.

Chapter 20

The narrow verifier and update rule

Mini-roadmap. The most important software interface is intentionally boring. The verifier receives the environment, admissible bank view, and certificate. It does not receive a compass score or creativity parameter.

Listing 20.1: Narrow verification interface

```
function VERIFY(env, bank, proposal):
    assert proposal.submitting_role in {P, R, I, C}
    assert proposal.environment_id == env.environment_id
    deps = bank.resolve_admissible(proposal.dependency_ids,
                                   scope=proposal.scope)
    checker = env.checker_versions[proposal.certificate_type]
    # submitting_role is provenance; it is not a truth input
    return checker.check(env,
                         semantic_kind=proposal.semantic_kind,
                         dependencies=deps,
                         certificate=proposal.certificate)
```

Listing 20.2: Verifier-gated update

```
function UPDATE(state, proposal, result):
    state.history.append(event(state.step, proposal, result))

    if result != ACCEPT:
        return state

    record = make_verified_record(proposal, result, state.step)
    state.bank.add(record)
    state.futurebank.mark_promoted_if_applicable(proposal)
    return state
```

20.1 Terminal classification

Listing 20.3: Terminal settlement

```
function TERMINAL_STATUS(env, bank, accepted_record):
    # Classify by verified semantics, not by who found the certificate.
```

```
    if accepted_record.semantic_kind == CONTRADICTION \
       and proves_contradiction(accepted_record):
        return INCONSISTENT
    if accepted_record.semantic_kind == TARGET_PROOF \
       and proves_target(env.target, accepted_record):
        return PROVED
    if accepted_record.semantic_kind == NEGATED_TARGET_PROOF \
       and proves_negated_target(env.target, accepted_record):
        return REFUTED
    if accepted_record.semantic_kind == INDEPENDENCE \
       and proves_independence(env.target, accepted_record):
        return INDEPENDENT
    return NONE
```

Contradiction is checked first here because the pseudocode assumes explosive logic. In a different logic, terminal precedence belongs in the verification environment. The classifier deliberately does not test the record's origin. If R submits an accepted target-proof certificate, the provenance remains R while the semantic status is PROVED. Also, first-certificate reporting should not be confused with a proof that the four evidence conditions are globally exclusive; Corollary 1.6 and Theorem 1.7 state the relevant distinction.

Chapter 21

Imagination, compass, and selection

Mini-roadmap. We now implement counterfactual search. IMAGINE may generate speculative nodes. COMPASS ranks them. SELECT chooses what to turn into an actual attempt. None of these functions can deposit a theorem directly.

Listing 21.1: Graded imagination

```
function IMAGINE(state, depth_budget, branch_budget):
    frontier = choose_future_frontier(state.futurebank)

    while budget_remains(depth_budget, branch_budget):
        if frontier.is_empty():
            break
        node = frontier.pop()
        actions = GENERATE_DEDUCTIVE_ACTIONS(state, node)

        for action in actions:
            child = APPLY_COUNTERFACTUALLY(node, action)

            if LOCAL_CHECK(action, node, child):
                child.trust_grade = LOCAL
            else:
                child.trust_grade = SPEC

            child.parent_ids = {node.id}
            child.bank_dependencies = trace_bank_dependencies(action)
            child.assumed_dependencies = trace_hypotheses(action)
            state.futurebank.add(child)
            frontier.push(child)

    return state.futurebank
```

Listing 21.2: Compass and controller

```
function COMPASS(state):
    for node in state.futurebank.expandable_nodes():
        node.compass_score = K1(state, node)

    actions = controller_actions(state)
    return argmax(actions, key=lambda a: K2(state, a))
```

```
function SELECT(state, controller_action, fallback=None):
    if controller_action.kind == "TRY_FUTURE_NODE":
        return materialize_certificate_proposal(controller_action.node)
    if controller_action.kind == "BACKFILL":
        return make_subgoal_proposal(controller_action.missing_dependency)
    if controller_action.kind == "SWITCH_ROLE":
        return next_agent_proposal(controller_action.role, state)
    if controller_action.kind == "FALLBACK" and fallback is not NONE:
        return fallback.next_proposal(state.bank)
    return NONE
```

21.1 Controlled structural escape

A controller can maintain a stagnation statistic based on repeated verifier failures, rank disagreement, or lack of bank growth. When tension is high it may raise novelty or switch regions. The control law is a search policy, not a theorem rule.

Chapter 22

The full AMLD loop

Mini-roadmap. This is the entire architecture in one place. The loop makes visible where each concept belongs and, just as importantly, where it does not belong.

Listing 22.1: Full AMLD loop

```
function AMLD_DECIDE(env, resource_budget):
    state = SearchState(
        bank = SEED_BANK(env),
        futurebank = new FutureBank(),
        history = [],
        reflection_state = initialize_reflection(env),
        controller_state = initialize_controller(),
        step = 0
    )

    fallback = initialize_complete_fallback_if_available(env)

    while resource_budget.allows_next_step():
        state.step += 1

        # 1. Counterfactual search may grow without changing theorem memory.
        state.futurebank = IMAGINE(
            state,
            depth_budget = controller_depth(state),
            branch_budget = controller_width(state)
        )

        # 2. Reflection may update correct self-descriptions used by control.
        state.reflection_state = REFLECT(state)

        # 3. Fair scheduling can reserve recurring fallback slots.
        if fallback is not NONE and \
           is_reserved_fallback_step(state.step, state.controller_state):
            proposal = fallback.next_proposal(state.bank)
        else:
            action = COMPASS(state)
            proposal = SELECT(state, action, fallback=fallback)

        if proposal is NONE:
            continue
```

```
        # 4. The verifier receives no compass score and no creativity score.
        result = VERIFY(env, state.bank, proposal)

        # 5. Only verifier-gated update can enlarge verified mathematical memory.
        state = UPDATE(state, proposal, result)

        # 6. An accepted terminal certificate settles the run.
        if result == ACCEPT:
            status = TERMINAL_STATUS(env, state.bank,
                                     state.bank.last_added_record())
            if status != NONE:
                return status, state

        # 7. Search learns from success and failure without rewriting proof semantics.
        state.controller_state = LEARN_CONTROL(state)

    return UNKNOWN, state
```

A reader should be able to point to every safety boundary in this program. IMAGINE can be wrong. COMPASS can be wrong. REFLECT can be incomplete. SELECT can be foolish. The controller can waste time. Those are search failures. The logical contract says that none of them may directly add a verified theorem.

A fallback engine is a search component, not a fifth semantic role. Any proposal it emits must still be wrapped in the same proposal type, with one submitting-role tag and one declared semantic certificate kind, before it reaches `VERIFY`.

Chapter 23

Safe compression and quotienting

> **Mini-roadmap.** Futurebanks grow rapidly. We therefore want to merge states. The dangerous shortcut is to merge two histories merely because their visible endpoint formulas look the same.

Let $M(h)$ be a compressed descriptor of Futurebank history h.

Definition 23.1 (Future-sufficient descriptor)**.** A descriptor M is *future-sufficient* for a declared task if

$$M(h_1) = M(h_2)$$

implies equality of all continuation information relevant to the task's future search, verification, scope, and required counting/provenance semantics.

Proposition 23.2 (Safe Futurebank quotient principle)**.** *If M is future-sufficient and any task-relevant path multiplicity or ancestry is retained separately, then histories with equal M-state may share downstream computation without changing the represented continuation structure relevant to that task.*

The difficult part is proving future-sufficiency.

Example 23.3 (Why endpoint equality fails)**.** Two branches may display the same current goal g while one branch depends on a temporary assumption h and the other does not. Their visible endpoint formula is identical, but their legal future deposits differ. Merging them while discarding scope is unsound.

Chess supplies the analogous warning: two identical piece arrangements can differ in castling rights or repetition history, so future legality can depend on the path.

> **Design principle.** Preserve provenance until you have proved which provenance is future-irrelevant.

Chapter 24

Software invariants and tests

Mini-roadmap. The theory suggests executable properties. A mature AMLD implementation should continuously test them, not merely describe them in prose.

Listing 24.1: Core invariant tests

```
property rejected_never_enters_bank:
    for every rejected proposal p:
        assert p.id not in bank.derived_record_ids

property every_derived_record_has_receipt:
    for r in bank.derived_records:
        assert r.verifier_receipt is not NONE

property verifier_policy_neutrality:
    choose same env, same bank, same semantic certificate s
    wrap s in proposals with different submitting roles
    vary futurebank, history, reflection, controller
    assert the intrinsic checker result on s is unchanged

property scope_is_dependency_closed:
    for r in bank.derived_records:
        assert all dependencies are admissible in r.scope

property futurebank_is_not_truth:
    for f in futurebank.nodes:
        assert f not in bank unless separately verifier-accepted

property quotient_preserves_required_provenance:
    for every merged quotient state q:
        assert task_required_ancestry(q) is reconstructible

property fair_fallback_progress:
    if fallback is enabled for an unbounded run:
        assert reserved schedule gives infinitely many effective slots
```

These properties are part of what it means to make the architecture scientific. They turn slogans into falsifiable software claims.

Exercise 24.1 (Invariant violation)**.** Suppose a developer modifies `VERIFY` to accept a certificate whenever either the formal checker accepts *or* the compass score exceeds 0.999. Identify every

theorem or invariant in this text that the change threatens. Give the smallest architectural repair.

Part VII

Worked Examples and Exercises

Chapter 25

Four tiny worked AMLDs

Roadmap. We revisit each terminal role using bank states and certificates explicitly. These examples are deliberately elementary so that no logical subtlety hides the architecture.

25.1 P settles q from p and $p \to q$

Environment:

$$\Gamma = \{p, p \to q\}, \qquad c = q.$$

Seed:

$$\mathsf{B}_0 = \{p, p \to q\}.$$

Proposal: P supplies a modus-ponens certificate. Verification accepts. Deposit q. Terminal classifier recognizes $\Gamma \vdash c$ and returns PROVED.

The Futurebank could have been empty. Imagination is an optimization layer, not a prerequisite for the logical meaning of this run.

25.2 R settles r from $\neg r$

Environment:

$$\Gamma = \{\neg r\}, \qquad c = r.$$

The seed bank already contains the formula required by the refutation certificate. Depending on the certificate format, R may submit a trivial reference-to-premise proof. The accepted terminal status is REFUTED.

25.3 I settles p over empty propositional theory

Environment:

$$\Gamma = \varnothing, \qquad c = p.$$

The I role supplies two valuations:

$$v_+(p) = \mathsf{true}, \qquad v_-(p) = \mathsf{false}.$$

A small model checker verifies that v_+ satisfies $\Gamma \cup \{p\}$ and v_- satisfies $\Gamma \cup \{\neg p\}$. Using the declared soundness rule, the metacertificate concludes that neither side follows from Γ.

This example demonstrates why I is not simply "wait for P and R to fail." It supplies a positive object.

25.4 C detects a broken foundation

Environment:

$$\Gamma = \{s, \neg s\}, \qquad c = t.$$

The contradiction agent certifies $\bot$. In an explosive environment the system returns INCONSISTENT without treating later proofs of t as informative about the intended target status.

Chapter 26

A line-by-line PA AMLD run: $1 + 1 = 5$

Roadmap. This chapter lets one small false arithmetic equation pass through the whole architecture. We write the AMLD as D, fix $\Gamma = \mathrm{PA}$, display the relevant seed BANK records, let all four roles act under a fair cyclic schedule, show what does and does not enter the FUTUREBANK and verifier history, and then inspect the winning R-certificate line by line. The example is intentionally elementary: the point is to make the machine visible.

26.1 The decider and its target

For this toy example, fix Peano arithmetic as the background theory and write

$$D(c) \in \{P, R, I, C, \mathsf{UNKNOWN}\}$$

for the operational result of the AMLD on a PA well-formed formula c. The role letter denotes the semantic settlement class of an accepted terminal certificate. Under the contradiction-dominance convention, an accepted C-certificate takes priority in an explosive inconsistent background. UNKNOWN remains an operational resource outcome rather than a fifth logical truth class.

Let S denote successor. Then

$$1 = S0, \qquad 2 = SS0, \qquad 5 = SSSSS0.$$

Take the target

$$c: \quad S0 + S0 = SSSSS0.$$

After normalizing the addition on the left, this becomes the equation suggested informally at the start:

$$SS0 = SSSSS0.$$

The AMLD will settle the original formula by producing a checked proof of its negation.

26.2 The relevant slice of $\Gamma = \mathrm{PA}$

The actual seed BANK contains the declared PA environment, not merely four formulas. For this proof, however, only a small slice is needed. Fix a standard presentation of PA (or a definitionally

equivalent presentation) in which the following formulas are available as axioms or already verified derived laws, together with ordinary equality rules:

$$\begin{aligned} A_1 &: \quad x + 0 = x, \\ A_2 &: \quad x + S(y) = S(x + y), \\ A_3 &: \quad Sx \neq 0, \\ A_4 &: \quad Sx = Sy \to x = y. \end{aligned}$$

The environment also supplies universal instantiation/substitution, equality symmetry, transitivity and congruence, and classical logical rules. Induction is not needed for this particular target.

Thus

$$\mathsf{B}_0 = \mathrm{seed}(\mathrm{PA}, \mathcal{E}).$$

The displayed formulas are only the portion of B_0 that the forthcoming certificate actually touches. Their origin is ENV, not one of the four agents.

26.3 A fair schedule chosen to make every role visible

A literal P, R, I, C round-robin would let R settle the target before the later roles necessarily do anything interesting. For pedagogy we instead use the cyclic order

$$P, I, C, R, P, I, C, R, \ldots$$

which is just as fair. If no terminal certificate appeared, every role would receive infinitely many turns.

Initialize

$$\Sigma_0 = (\mathsf{B}_0, \mathsf{F}_0, \mathsf{H}_0, \mathsf{Ref}_0, \mathrm{Ctrl}_0), \qquad \mathsf{F}_0 = \varnothing, \qquad \mathsf{H}_0 = \varnothing.$$

We now follow one cycle.

26.4 Turn 1: P normalizes the left side

P is trying to prove c. It does not succeed, but it notices an ordinary arithmetic simplification that is useful to everyone.

Instantiate A_2 with $x = S0$ and $y = 0$:

$$S0 + S0 = S(S0 + 0).$$

Instantiate A_1 with $x = S0$:

$$S0 + 0 = S0.$$

By congruence of S,

$$S(S0 + 0) = SS0.$$

By transitivity,

$$L_1 : \quad S0 + S0 = SS0.$$

P submits L_1 as

$$(P, (\mathsf{INTERMEDIATE\_THEOREM}, \kappa_1)).$$

The intrinsic verifier checks the arithmetic/equality derivation and returns `ACCEPT`. Therefore

$$\mathsf{B}_1 = \mathsf{B}_0 \cup \{L_1\}.$$

The record for L_1 has origin P, semantic kind $\mathsf{INTERMEDIATE\_THEOREM}$, scope Γ, and dependencies pointing back to the relevant PA seed records.

This is an important AMLD event: P has not proved the target, yet its work survives and may help an opposing role.

26.5 Turn 2: I considers independence

I must not infer independence from the absence of a proof. A natural speculative strategy is to seek a model of

$$\mathrm{PA} \cup \{c\}$$

and a model of

$$\mathrm{PA} \cup \{\neg c\}.$$

At this moment the role may place a plan of that form into the speculative FUTUREBANK:

$$f_I : \quad \text{“seek an admissible model-pair certificate for } c\text{.”}$$

No such terminal certificate has been verified, so the BANK does not change:

$$\mathsf{B}_2 = \mathsf{B}_1.$$

Verifier/search history records that I considered or attempted this route. The distinction is deliberate:

$$\text{interesting search information} \neq \text{proved mathematics}.$$

26.6 Turn 3: C contributes a theorem without finding inconsistency

C searches for a contradiction in the background. It does not produce a certificate of $\mathrm{PA} \vdash \bot$. But while examining the successor axioms, it obtains a useful instance of A_3 by substituting $x = SS0$:

$$L_2 : \quad SSS0 \neq 0.$$

The verifier accepts the certificate for L_2, so

$$\mathsf{B}_3 = \mathsf{B}_2 \cup \{L_2\}.$$

The origin field says C. The semantic kind is still an ordinary intermediate theorem. By Corollary 8.1, R may use it if its type and scope are admissible.

Again the role did not achieve its own terminal objective, yet its verified work remains useful.

26.7 The FUTUREBANK just before the winning turn

At this point a small imagined search structure might contain branches such as

f_P : continue searching for a proof of c,
f_I : seek a model pair for independence,
f_C : continue testing the PA background for contradiction,
f_R : use L_1 to reduce the target to $SS0 = SSSSS0$ and strip successors.

A COMPASS is permitted to rank f_R highly because it now has verified dependencies L_1 and L_2. That ranking does not itself prove anything. It merely helps choose the next attempt.

26.8 Turn 4: R submits the terminal refutation

R wants a certificate of

$$\neg(S0 + S0 = SSSSS0).$$

The following natural-deduction presentation makes the certificate readable line by line. A concrete PA implementation could encode the same reasoning in another proof calculus.

Line	Formula	Justification
1	$S0 + S0 = SSSSS0$	temporary assumption c
2	$S0 + S0 = SS0$	BANK lemma L_1
3	$SS0 = S0 + S0$	symmetry of line 2
4	$SS0 = SSSSS0$	transitivity, lines 3 and 1
5	$S0 = SSSS0$	successor injectivity A_4, line 4
6	$0 = SSS0$	successor injectivity A_4, line 5
7	$SSS0 = 0$	symmetry of line 6
8	$SSS0 \neq 0$	BANK lemma L_2
9	$\bot$	lines 7 and 8
10	$\neg(S0 + S0 = SSSSS0)$	discharge line 1

Thus R submits a proposal whose semantic kind is NEGATED_TARGET_PROOF. Its dependency DAG contains a lemma originating with P, a lemma originating with C, and seed records originating with ENV. The verifier checks the certificate independently of those origins and accepts it.

The terminal BANK state is therefore

$$\mathsf{B}_4 = \mathsf{B}_3 \cup \{\neg(S0 + S0 = SSSSS0)\},$$

and the decider returns

$$\boxed{D(c) = R = \mathsf{REFUTED}.}$$

The run halts. I's speculative model-pair branch never becomes theorem memory, and C never claimed that PA itself was inconsistent.

26.9 The BANK ledger for the whole run

The logical growth can be summarized compactly:

State	New verified content	Origin	Semantic kind
B_0	PA seed records, including the displayed relevant axioms	ENV	AXIOM/HYPOTHESIS
B_1	$L_1 : S0 + S0 = SS0$	P	INTERMEDIATE_THEOREM
B_2	no new verified record	—	—
B_3	$L_2 : SSS0 \neq 0$	C	INTERMEDIATE_THEOREM
B_4	$\neg(S0 + S0 = SSSSS0)$	R	NEGATED_TARGET_PROOF

The corresponding verifier/search history H additionally records the accepted receipts for L_1, L_2, the terminal R-certificate, and I's unsuccessful or merely speculative work. This is exactly why H is not the BANK.

26.10 What the example teaches

1. The target is settled by R, but R does not have to discover every lemma in its own certificate.
2. A P-discovered theorem can help refute the target; a C-discovered theorem can do the same.
3. The winning semantic role is determined by what the accepted certificate proves, not by the provenance of every dependency.
4. FUTUREBANK branches may guide attention without entering theorem memory.
5. A tiny false arithmetic equation already exercises Γ, seed records, shared BANK, provenance, agents, imagination, COMPASS selection, verification, and terminal classification.

Corollary 26.1 (Cross-role terminal assembly)**.** *Under origin-neutral admissibility, an accepted terminal certificate may depend on verified BANK records originating from the environment and from any mixture of the roles P, R, I, C. If the dependencies are semantically admissible and the terminal certificate verifies, the terminal settlement meaning is determined by the certificate's semantic kind, not by the origin labels on its dependency DAG.*

Proof. Corollary 8.1 makes admissible verified records reusable across roles, while Corollary 3.8 makes terminal semantics independent of the discovering role. Combining the two facts gives the result. □

Lemma 26.2 (Normalization-refutation lemma)**.** *In classical first-order logic with equality, if*

$$\Gamma \vdash t = u \qquad \textit{and} \qquad \Gamma \vdash u \neq v,$$

then

$$\Gamma \vdash t \neq v.$$

Proof. Assume $t = v$. From $t = u$, symmetry gives $u = t$, and transitivity with $t = v$ yields $u = v$, contradicting $u \neq v$. Discharge the temporary assumption. □

The PA run is an instance with

$$t = S0 + S0, \qquad u = SS0, \qquad v = SSSSS0.$$

The BANK lemma L_1 supplies the normalization $t = u$; the rest of the R-certificate supplies $u \neq v$.

Exercise 26.3 (Replay the PA refutation)**.** Repeat the run with target

$$c' : \quad SS0 = SSSSS0$$

so that no addition-normalization lemma is needed. Give the shortest readable R-certificate you can using successor injectivity and the axiom that no successor equals zero. Then explain which architectural features disappear from the example when L_1 is no longer needed.

Chapter 27

A worked Futurebank example

> **Mini-roadmap.** We now add imagination to a toy proof. The example shows the difference between seeing a route and owning the lemmas required to travel it.

Suppose Γ contains rules sufficient for

$$\lambda \to \mu, \qquad \mu \to c,$$

but λ has not yet been proved. A speculative Futurebank may contain the conditional route

$$[\text{suppose } \lambda] \to \mu \to c.$$

This is useful because it identifies λ as a strategically valuable subgoal. Yet neither μ nor c may be deposited globally from this branch while λ remains hypothetical.

The controller can backfill:

1. search for a certificate of λ from Γ;
2. if accepted, deposit λ;
3. apply the verified rule to obtain μ and deposit it after checking;
4. derive c and terminate after checking.

The imagined route functions like a chess variation: it tells the machine what a successful continuation would look like. Verification turns the variation into a realized proof history.

Chapter 28

Exercises

Mini-roadmap. These exercises are not research conjectures. They test whether the architecture is understood. Solution sketches follow in the next chapter.

Exercise 28.1 (Axiom versus theorem)**.** Why should a seed record for $\gamma \in \Gamma$ not be labeled simply "proved"? Give a data schema that distinguishes an admitted premise from a derived theorem while allowing both to be used as premises when appropriate.

Exercise 28.2 (Independence certificate)**.** In classical propositional logic with $\Gamma = \{p \vee q\}$ and target p, give two valuations that witness that p is independent of Γ. Explain what the verifier must check.

Exercise 28.3 (Futurebank contamination)**.** A speculative node contains the statement L because the controller asked "what if L were true?" A programmer inserts L into the BANK to make downstream search faster and adds a flag `speculative=true`. Is this safe if ordinary deduction routines can still read L? Explain.

Exercise 28.4 (Fairness)**.** Suppose complete fallback is invoked on steps $1, 2, 4, 8, 16, \ldots$. Is the schedule fair to fallback in the sense required by Theorem 16.1? Does the density of fallback steps need to stay bounded away from zero?

Exercise 28.5 (Unsafe quotient)**.** Construct two proof histories that end with the same visible goal but should not be merged because one carries an extra local assumption. What extra information would make a quotient descriptor safer?

Exercise 28.6 (Role cooperation)**.** Describe a run in which I discovers a model or structural fact that leads P to a useful object-level lemma, but I never produces a final independence certificate. What should be stored in BANK and what should remain in verifier/search history?

Chapter 29

Solution sketches

Exercise 3.5. Both proposals have submitting-role tag I because I discovered and submitted them. The ordinary lemma should carry an object-level intermediate semantic kind such as INTERMEDIATE_THEOREM; if accepted, it enters theorem memory with origin I and its ordinary logical scope/dependencies. The later proof of c carries semantic kind TARGET_PROOF. Its accepted BANK record still has origin I, but its terminal settlement role is P and it licenses PROVED. Provenance answers who found it; semantics answers what it proves.

> **Mini-roadmap.** The sketches emphasize the conceptual point of each exercise rather than presenting one mandatory implementation.

Exercise 1.10. (a) computational stopping only; (b) both a computational terminal event and mathematical settlement if the checker is sound; (c) computational stopping only; (d) mathematical settlement and normally terminal computation; (e) neither mathematical settlement nor necessarily a justified exhaustive claim.

Exercise 2.2. B_0 contains admitted records for a, $a \to b$, and $b \to d$. The three-stage bank history can be written $\mathsf{B}_0, \mathsf{B}_1, \mathsf{B}_2$: start with admitted records for a, $a \to b$, and $b \to d$; verify and deposit b to obtain B_1; then verify and deposit d from b and $b \to d$ to obtain B_2. The three Γ records are admitted premises; b and d are derived.

Exercise 6.2. The certificate for u establishes at most $\Gamma \cup \{h\} \vdash u$. Re-labeling it as $\Gamma \vdash u$ silently discharges an assumption. Store an explicit scope or assumption set in each record and require the verifier to check dependency closure.

Exercise 8.2. Many answers work. For example, while attempting $\neg c$, R may prove a general algebraic identity L from Γ. If L is globally scoped and verified, P may use it later. Origin is provenance, not ownership.

Exercise 9.1. Position $\leftrightarrow$ proof state; legal move $\leftrightarrow$ locally admissible deduction; principal variation $\leftrightarrow$ favored proof route; transposition $\leftrightarrow$ two histories reaching a future-equivalent proof

state; evaluation ↔ compass score; played move ↔ selected and committed verified action. The analogy is imperfect because ATP branches may intentionally contain unproved hypothetical lemmas.

Exercise 12.2. Use a DAG $\lambda \to \mu \to c$. Speculative exploration may reason conditionally from both unproved nodes. Promotion requires an accepted certificate of λ, followed by an accepted certificate of μ using the now-verified λ, followed by the target.

Exercise 15.3. Examples: a simple nonreflective chess-like proof search (high Futurebank, low formal self-description); a tagged trivial loop proving one self-description (low Futurebank, high self-description); an ordinary simple prover (low/low); and a reflective Futurebank controller (high/high). This shows that the capabilities are not one scalar.

Exercise 24.1. The change violates policy-neutral verification, noncontamination, and the principle that compass confidence cannot authorize theorem status. Repair: remove compass score from the acceptance disjunction and keep it only in selection/ranking.

Exercise 28.1. Use a field such as `kind=AXIOM|HYPOTHESIS|DERIVED`. Seed records obtain their authority from the environment declaration, while derived records require verifier receipts and dependencies.

Exercise 28.2. For $p \vee q$, choose $v_1(p) = T, q = F$ and $v_2(p) = F, q = T$. Both satisfy Γ; one satisfies p, the other $\neg p$. The verifier checks the valuations and the metatheoretic soundness rule that converts the model pair into nonderivability on both sides.

Exercise 28.3. It is unsafe if ordinary deduction can consume the flagged record as a premise. A flag is not quarantine unless admissibility routines enforce it. Keep the item in F_{spec} or require a type system that makes speculative records unusable by verified deduction.

Exercise 28.4. Yes. The powers-of-two schedule gives infinitely many fallback steps, so it is fair in the theorem's weak sense. Its asymptotic density is zero; positive density is not required by Theorem 16.1, though performance may be poor.

Exercise 28.5. One history may end at goal g under scope Γ and another at the same displayed g under $\Gamma \cup \{h\}$. A safer descriptor includes active assumptions, admissible dependency set, role/metatheory context, and other continuation-relevant state.

Exercise 28.6. Store any independently verified, correctly typed structural fact that is admissible for later use. Store unsuccessful independence attempts, heuristic scores, and rejected metacertificates in history rather than theorem memory.

Exercise 18.12. The sorted probabilities are already $(0.40, 0.25, 0.15, 0.10, 0.10)$, so $G_1 = 0.40$, $G_2 = 0.65$, and $G_3 = 0.80$. With no new side information, any one-, two-, or three-candidate strategy can cover at most that much prior probability mass. More internal computation can reorder candidates, but it cannot make a three-element guessed set contain more than the top-three mass.

Exercise 18.13. The server-side retry counter is consumed only by interactions with the live server. A separate local verifier need not consult that state, so the number ten has no direct force there. The relevant defenses become the password distribution and the cost of each local verification attempt, which is why modern password hashing, unique salts, and suitable work factors matter.

Exercise 18.14. One suitable target is $c : \forall z[\mathsf{Impl}(z) \Rightarrow (P(z) \wedge F(z))]$. The refuter R needs a checked event z^* for which the implementation accepts but at least one required predicate is false. Such a witness is a finite counterexample certificate.

Exercise 18.15. For example, a system that permanently locks an account after one failed attempt gives an unauthorized guesser almost no online budget, but any person who can submit one incorrect attempt for a known username may also deny the legitimate user access. The objective therefore includes both confidentiality/authentication risk and availability. Smaller N improves one dimension only under additional assumptions.

Exercise 26.3. Starting from $SS0 = SSSSS0$, successor injectivity gives $S0 = SSSS0$, then $0 = SSS0$. Symmetry gives $SSS0 = 0$, contradicting the PA axiom instance $SSS0 \neq 0$. Discharging the initial assumption yields $\neg(SS0 = SSSSS0)$. Because the target is already normalized, the example no longer needs the P-origin lemma L_1 or the normalization-refutation step; it therefore shows less cross-role BANK cooperation than the $1 + 1 = 5$ version.

Exercise 17.10. (a) ${}^*\mathbb{N}$ contains both standard and nonstandard hypernaturals, so membership alone does not imply unlimitedness. (b) Any ordinary PA proof has a standard finite code n, and unlimited H satisfies $n < H$. (c) For every standard N, the standard number $N + 1$ lies beyond it; Theorem 17.6 sharpens this by showing that some standard PA theorem has no proof code below any prescribed finite cutoff. (d) A discovered transferred $P/R/C$ proof witness implies that an ordinary finite proof exists by Theorem 17.1. In the I case, the completed hyperfinite nonfinding plus the external fact that H is unlimited supports the external classification, but the unlimited cutoff itself does not become an ordinary finite cutoff certificate.

Part VIII

Research Frontier

Chapter 30

Conjectures

Roadmap. Only now, after the architecture is defined and exercised, do we state research conjectures. Each is deliberately separated from the proved invariants above. The following are hypotheses about search efficiency, not changes to proof semantics.

Conjecture 30.1 (Reflective navigation advantage)**.** For some nontrivial families of proof-search problems, a Futurebank-reflective controller with correct access to selected facts about its own search state can reduce expected search cost relative to an otherwise matched Futurebank/compass architecture lacking those reflective features, while leaving the intrinsic verifier unchanged.

Conjecture 30.2 (Graded imagination advantage)**.** For some hard proof-search distributions, allowing quarantined speculative Futurebank expansion before expensive final verification can reduce expected total search cost relative to an architecture that fully verifies every imagined edge before deeper expansion, while preserving verifier-gated noncontamination.

Conjecture 30.3 (Tension-triggered structural escape)**.** On some problem distributions with deceptive local guidance, a controller that increases structural novelty in response to sustained disagreement between compass ranking and verifier outcomes can outperform a matched controller with a fixed novelty schedule.

Conjecture 30.4 (Backfill advantage)**.** On some theorem families with expensive auxiliary lemmas, conditional exploration followed by selective dependency backfilling can reduce expected work relative to proving every candidate auxiliary lemma before exploring its downstream consequences.

Conjecture 30.5 (Moderate reflection optimum)**.** For at least some finite-resource AMLD architectures, an intermediate degree of formal self-description can yield better search efficiency than either no reflection or substantially more expensive reflection, because useful controller information is obtained before metareasoning overhead dominates.

The last conjecture is intentionally weaker than the slogan "Level 2 is always optimal." No theorem in this book establishes a universal creativity optimum at Level 2. Any such universal claim would be easy to attack by changing the cost model or constructing a problem on which reflection is irrelevant.

30.1 Password-domain research conjectures

The proved results in the password special case were deliberately information-theoretic or logical. The following stronger claims concern search efficiency and security-control design and should be treated as conjectures until tested under preregistered models.

Conjecture 30.6 (Audit-over-secret advantage)**.** On some realistic classes of well-rate-limited authentication systems with low top-N exposure, allocating AMLD-style imagination and verification primarily to specification-conformance, recovery-route, and control-policy auditing yields greater defensive value per unit computation than allocating the same resources primarily to direct password-candidate ranking.

Conjecture 30.7 (Cross-route shared-BANK advantage)**.** For authentication systems with multiple sufficient routes such as password login, recovery, remembered sessions, and second-factor flows, a provenance-preserving shared BANK of verified invariants and counterexample lemmas can reduce expected audit cost relative to matched isolated audits of each route, particularly for defects that arise only from route interaction.

Conjecture 30.8 (Adaptive flood-control frontier)**.** For some threat and usability distributions, an adaptive flood-control policy using verified account and risk state can achieve a strictly better tradeoff between unauthorized-acceptance probability and legitimate-user denial than every fixed hard-lockout policy in a specified comparison class.

Conjecture 30.9 (Verifier-history repair advantage)**.** For some authentication-audit distributions, retaining rejected defect hypotheses and failed repair attempts in verifier history, while keeping them out of the verified BANK, can improve subsequent repair selection without increasing false certification of defects or fixes.

30.2 Tempting stronger claims that should already be rejected

The following are not retained as conjectures because simple counterexamples undermine them:

- **Deeper Futurebanks can never hurt.** False under finite memory or time: extra branches can consume the budget before a useful shallow branch is attempted.
- **More speculation can only help.** False when speculation overwhelms ranking, memory, or backfill resources.
- **Same endpoint means safe merge.** False when scope or other path-dependent continuation information differs.
- **A sufficiently good compass makes verification unnecessary.** False as an architectural claim: ranking confidence and certificate validity are different predicates.
- **Long joint failure of P and R establishes independence.** False: search failure is not a nonderivability certificate.
- **A certificate is valid because it came from the “right" agent.** False: under the narrow interface, agent identity is provenance; validity is determined by semantic certificate content, scope, dependencies, environment, and checker.

- **Higher formal self-awareness necessarily improves imagination.** False by Proposition 14.1; the capabilities are logically distinct.

Chapter 31

LAB: How to try to refute the conjectures

Roadmap. This Lab is adversarial by design. The reader's job is not to make the conjectures look good. It is to give them a fair chance to fail. We first explain the general experimental protocol, then give a refutation roadmap for every conjecture above.

31.1 First lesson: a vague existential conjecture is hard to refute experimentally

Statements such as "there exist some problem families on which method A helps" are mathematically weak and scientifically slippery. Failure on one benchmark does not refute the existence of another benchmark where A helps.

Therefore the Lab converts each broad research conjecture into a preregistered benchmark-level hypothesis. Fix:

- a benchmark distribution D;
- a train/dev/test split chosen before the final comparison;
- identical verifier V and verification environment format;
- identical total CPU/GPU/time or verifier-call budget;
- fixed random seed protocol;
- a primary metric, such as solved fraction, verifier calls to settlement, wall-clock time, or area under a solved-versus-budget curve;
- a minimum practically meaningful effect size δ;
- a statistical decision rule.

Then ask a finite question. For example:

> On preregistered test distribution D, does reflective controller A improve median verifier calls by at least δ relative to matched controller B?

That claim can fail cleanly.

31.2 Universal Lab protocol

For every conjecture:

1. **Freeze correctness.** Use exactly the same verifier binary/version and certificate semantics in every condition.
2. **Change one architectural feature.** Do not compare two systems that differ in five uncontrolled ways.
3. **Pair runs.** Run both conditions on the same problems and, where stochasticity exists, matched random seeds.
4. **Record the whole trace.** Keep BANK growth, verifier calls, Futurebank node counts, rejected proposals, role allocations, fallback steps, and terminal status.
5. **Use a resource curve.** A method that wins only because it received more computation has not established the intended advantage.
6. **Attempt counterexample construction.** In addition to random benchmarks, design adversarial instances specifically meant to exploit the proposed mechanism's overhead.
7. **Publish negative results.** A clean refutation is scientific progress.

31.3 Lab A: Reflective navigation advantage

Ablation. Compare the same Futurebank, compass, verifier, and fallback with reflection enabled versus disabled. Reflection-enabled search may read certified self-state facts; reflection-disabled search receives matched non-self-referential control inputs where possible.

Refutation target. Preregister a benchmark-relative claim such as a minimum δ reduction in verifier calls. Refute that claim if the paired effect fails the threshold or moves significantly in the wrong direction.

Adversarial cases. Use problems where the correct proof is nearly immediate and reflection overhead is pure cost; use problems where self-state statistics are intentionally uninformative; use a controller that can already infer equivalent information without formal reflection.

31.4 Lab B: Graded imagination advantage

Ablation. Condition G permits $\mathsf{F}_{\mathrm{spec}}$ expansion; condition E requires each edge to be fully certified before deeper expansion. Keep branch generator and ranking model matched.

Refutation target. Show that speculative expansion does not reach the preregistered efficiency threshold, or that memory/branch explosion increases total cost enough to erase the benefit.

Adversarial cases. Construct domains in which almost every speculative auxiliary lemma is false or unprovable, so that deep counterfactual exploration spends effort on dead branches.

31.5 Lab C: Tension-triggered structural escape

Ablation. Compare an adaptive novelty controller using compass/verifier disagreement with a fixed novelty schedule matched for average novelty budget.

Refutation target. Find a preregistered deceptive-guidance benchmark where the tension signal does not improve performance, or where it systematically triggers escapes just before the local search would have succeeded.

Adversarial cases. Create noisy verifier-failure patterns that are not evidence of local trapping. If tension confuses ordinary hard progress with failure, it may thrash between regions.

31.6 Lab D: Backfill advantage

Ablation. Condition B explores conditional downstream utility before proving auxiliary lemmas. Condition P requires proof-first certification of each candidate lemma. Use the same candidate generator.

Refutation target. Show that speculative downstream analysis costs more than the proof effort it saves, or that it repeatedly selects high-utility-looking lemmas that turn out to be impossible to backfill.

Adversarial cases. Generate many seductive hypothetical lemmas that would trivialize the target if true but are false or extremely hard to prove.

31.7 Lab E: Moderate reflection optimum

Ablation. Implement at least three reflection budgets: none, moderate, and high. If using the Level terminology, make the cost of obtaining self-descriptive statements explicit rather than assuming it is free.

Refutation target. Refute a benchmark-specific "middle is best" claim by showing monotone improvement with more reflection, monotone degradation, or no meaningful difference. To attack the stronger slogan "Level 2 is universally optimal," a single well-specified counterexample architecture in which another level strictly wins is enough.

31.8 Lab F: Audit-over-secret advantage

Ablation. Fix an authorized synthetic authentication model with a hard or effective attempt budget N. Give condition S most discretionary computation to candidate ranking and condition A the same budget for formal conformance, recovery-route, and policy-defect search. Keep the underlying model and checker fixed.

Refutation target. Reject the benchmark-level advantage claim if condition A fails to find more verified defensive defects, fails to certify more relevant invariants, or consumes enough overhead to erase the preregistered value threshold.

31.9 Lab G: Cross-route shared-BANK advantage

Ablation. Compare isolated route audits with an otherwise matched shared-BANK audit. The shared condition may reuse only verified, correctly scoped invariants and lemmas.

Refutation target. Show that sharing provides no measurable audit benefit, or that bookkeeping and overly broad reusable facts create enough overhead to make isolation better.

31.10 Lab H: Adaptive flood-control frontier

Ablation. Compare preregistered fixed lockout policies against an adaptive throttling controller on the same simulated legitimate-user and adversarial-attempt distributions.

Refutation target. Plot unauthorized-acceptance risk against legitimate-user denial and latency. Refute the finite benchmark claim if the adaptive policy is Pareto-dominated by a fixed policy or fails the declared improvement margin.

31.11 Lab I: Verifier-history repair advantage

Ablation. Compare two repair-search systems that share the same verified BANK. One may consult structured history of rejected defect hypotheses and failed repairs; the other may not.

Refutation target. Reject the advantage claim if history fails to improve verified repair rate or if history-driven search repeatedly wastes budget on stale failure modes.

31.12 What counts as a successful refutation?

There are two very different standards:

1. A **mathematical universal claim** can be refuted by one valid counterexample satisfying its hypotheses.
2. An **empirical benchmark claim** is refuted according to its preregistered finite decision rule.

Do not pretend that a failed experiment disproves an existential mathematical statement over all possible problem families. Instead, write benchmark-level hypotheses strongly enough that the data are allowed to say "no."

31.13 Minimal experiment pseudocode

Listing 31.1: Paired ablation harness for any Lab conjecture

```
function RUN_ABLATION(benchmark, system_A, system_B, budgets, seeds):
    results = []

    for problem in benchmark.test_problems:
        for seed in seeds:
```

```
            for budget in budgets:
                trace_A = run(system_A, problem, seed, budget)
                trace_B = run(system_B, problem, seed, budget)

                assert trace_A.verifier_version == trace_B.verifier_version
                assert trace_A.problem_environment == trace_B.problem_environment

                results.append({
                    "problem": problem.id,
                    "seed": seed,
                    "budget": budget,
                    "A": summarize(trace_A),
                    "B": summarize(trace_B)
                })

    return paired_analysis(results, preregistered_primary_metric)
```

The final scientific habit is perhaps the most important one in the book: a search idea should earn its place by surviving both the verifier and the experiment appropriate to it.

Closing roadmap

The architecture can now be remembered in nine sentences.

1. Γ is the admitted starting point, and the BANK begins by recording it explicitly.
2. P, R, I, C are four principal search roles. Every agent proposal has exactly one submitting-role tag, while certificate semantic kind is a separate coordinate.
3. A terminal certificate has a designated settlement role determined by its verified semantics; its submitting role is provenance and need not match that settlement role.
4. The BANK is shared, typed, provenance-preserving verified memory, so intermediate mathematics can cross agent boundaries safely.
5. The FUTUREBANK contains represented possible proof futures, including quarantined hypothetical lemmas, much as chess search contains moves that have not been played.
6. A deductive move changes a proof-search state; imagination recursively explores such moves, while planning and the compass decide which imagined route deserves real computation.
7. Formal self-description may improve control, but it is a separate axis from imagination and may never redefine the verifier.
8. AMLD is a general architecture whose applications may include mathematical settlement, software and protocol verification, engineering safety, formalized scientific reasoning, and many other certificate-bearing domains; authentication and passwords are only one worked special case showing how domain-specific information and external controls can bound search.
9. The verifier is the fixed gate between "this might work" and "this is now part of our mathematics."

References and live links

[1] Max Tegmark, "Is 'the theory of everything' merely the ultimate ensemble theory?", *Annals of Physics* 270 (1998), 1–51. ArXiv: https://arxiv.org/abs/gr-qc/9704009.
DOI: https://doi.org/10.1006/aphy.1998.5855.

[2] Claude E. Shannon, "Programming a Computer for Playing Chess", *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, Series 7, 41(314) (1950), 256–275.
DOI: https://doi.org/10.1080/14786445008521796.
Computer History Museum copy: https://www.computerhistory.org/chess/doc-431614f453dde/.

[3] Metamath Proof Explorer, Theorem `2p2e4`, "Two plus two equals four." https://us.metamath.org/mpeuni/2p2e4.html.
Metamath Proof Explorer home page: https://us.metamath.org/mpeuni/mmset.html.

[4] Geoff Sutcliffe and the TPTP project, "TPTP and TSTP Quick Guide." https://tptp.org/UserDocs/QuickGuide/.

[5] TPTP project, "The TPTP Language" and SZS status documentation. https://tptp.org/UserDocs/TPTPLanguage/TPTPLanguage.shtml.

[6] Max Wisniewski, *Agent-based Blackboard Architecture for a Higher-Order Theorem Prover*, Master's thesis, Freie Universität Berlin, 2014. Public PDF: live PDF.

[7] Jan Jakubův and Josef Urban, "ENIGMA: Efficient Learning-based Inference Guiding Machine", 2017. ArXiv: https://arxiv.org/abs/1701.06532.

[8] Brian Tenneson, *Automated Logical Deciders: Certificate-Indexed Halting, Multiagent Search, Provenance-Aware Shared Lemma Pools, and Machine-Learned Guidance*, research supplement, August 2026. Snapshot repository: https://github.com/btenneson/Automatic-Logical-Deciding-Snapshot-August-2-2026-/.

[9] Brian Tenneson, *Depths of a Simulation & Formalized Self-Awareness*, merged edition, July 2026. Author manuscript. Project repository: https://github.com/btenneson/Depths-of-a-Simulation.

[10] Brian Tenneson, *Fund the Proof Machines—Guard the Capabilities That Can Become Weapons*, Substack essay, 2026. The essay argues for broad investment in open, independently verifiable ATP research while treating operationally dangerous capabilities as a separate governance problem; its dual-use examples include mathematics, software verification, authentication, cryptographic protocols, and cyber-physical systems. Live link: https://btenneson2301.substack.com/p/fund-the-proof-machinesguard-the.

[11] National Institute of Standards and Technology, *Digital Identity Guidelines: Authentication and Authenticator Management*, NIST SP 800-63B-4, July 2025. Live HTML: https://pages.nist.gov/800-63-4/sp800-63b.html.
DOI: https://doi.org/10.6028/NIST.SP.800-63B-4.

[12] OWASP Cheat Sheet Series, "Authentication Cheat Sheet." Live guide: OWASP Authentication Cheat Sheet.

[13] OWASP Cheat Sheet Series, "Password Storage Cheat Sheet." Live guide: OWASP Password Storage Cheat Sheet.

[14] H. Jerome Keisler, *Foundations of Infinitesimal Calculus*, online monograph, 2007 (online edition updated June 2022). University of Wisconsin–Madison author page and live monograph link: https://people.math.wisc.edu/ hkeisler/foundations.html.

Editorial provenance note

The AMLD roles, shared BANK, provenance emphasis, FUTUREBANK, compass/control architecture, verifier separation, and formalized self-awareness program are developed from Brian Tenneson's prior manuscripts and research discussions. This textbook edition was substantially reorganized, expanded, formalized, and edited with AI assistance. Because the scope and prose have changed considerably from earlier preview drafts, attribution percentages stated in those earlier drafts should not be assumed to apply unchanged to this edition.